



Trees — Complete Placement & CP Guide

A comprehensive guide to tree data structures and algorithms for SDE interviews, online assessments, and competitive programming.

Table of Contents

1. [Binary Tree Basics](#)
 2. [Tree Traversal](#)
 3. [Preorder Traversal](#)
 4. [Inorder Traversal](#)
 5. [Postorder Traversal](#)
 6. [Level Order Traversal](#)
 7. [Height / Depth of Tree](#)
 8. [Diameter of Tree](#)
 9. [Balanced Tree Check](#)
 10. [Lowest Common Ancestor](#)
 11. [Root-to-Leaf Paths](#)
 12. [Path Sum](#)
 13. [Binary Search Tree](#)
 14. [Insert / Search / Delete in BST](#)
 15. [Validate BST](#)
 16. [Serialize / Deserialize Tree](#)
 17. [Construct Tree from Traversals](#)
 18. [Vertical Order Traversal](#)
 19. [Boundary Traversal](#)
 20. [Morris Traversal](#)
 21. [Binary Lifting for LCA](#)
 22. [Tree DP](#)
 23. [Heavy-Light Decomposition](#)
 24. [Centroid Decomposition](#)
 25. [Euler Tour + Segment Tree](#)
 26. [Link-Cut Tree](#)
-

1. Binary Tree Basics

1.1 Overview

A **binary tree** is a hierarchical data structure where each node has at most **two children**, called the **left child** and **right child**. It is one of the most fundamental non-linear data structures in computer science. Unlike arrays, linked lists, stacks, and queues that are linear, a binary tree organizes data in a parent-child relationship, forming a rooted tree.

Each node contains:

- **Data** (the value stored)
- **Left pointer** (points to left child, or NULL)
- **Right pointer** (points to right child, or NULL)



1.2 Intuition

Simple explanation: A binary tree is like a family tree where every person can have at most two children. You start from the root (the oldest ancestor) and can move downward.

Analogy: Think of a company's organization chart. The CEO (root) has two VPs (children). Each VP has two directors, and so on. Some people may have zero, one, or two direct reports.

Why it works: Binary trees enable efficient **divide-and-conquer**. By splitting data into two halves at each level, operations like search, insert, and delete can be done in $O(\log n)$ time in balanced trees. The parent-child relationship also allows natural recursive processing.

1.3 When to Use It

- Data has a **hierarchical** relationship (filesystem, DOM, organizational chart)
- Need **efficient search/insert/delete** operations
- Need to represent **expressions** (AST, arithmetic expressions)
- Problems involving **recursive subdivision** (merge sort tree, segment tree)
- Need **ordered traversal** (BST, inorder gives sorted order)

Common triggers:

- "Hierarchical data"
- "Parent-child relationship"
- "Recursive structure"
- "Divide into two parts"

1.4 When Not to Use It

- Data is **flat** or **linear** — use arrays or linked lists
- Only need **fast key lookup** without ordering — use hash tables ($O(1)$ average)
- Tree would become **highly skewed** (like a linked list) — consider balanced variants
- **Memory is severely constrained** — every node has pointer overhead (2 pointers per node)
- Only need **min/max** operations — use a heap (simpler)
- **Very small datasets** — linear structures may outperform due to cache locality

1.5 Core Concepts

Concept	Definition	Why It Matters
Root	Topmost node, no parent	Entry point for all operations
Node	Element containing data + children pointers	Basic building block
Edge	Connection between parent and child	Defines tree structure
Leaf	Node with no children	Terminal nodes, base cases
Subtree	A node and all its descendants	Recursive decomposition
Height	Longest path from node to a leaf	Tree balance, complexity
Depth	Distance from root to node	Level of a node
Level	All nodes at same depth	BFS traversal layers
Full Tree	Every node has 0 or 2 children	Structural property
Complete Tree	All levels filled except possibly last, which is left-filled	Array representation
Perfect Tree	All internal nodes have 2 children, all leaves at same level	Maximum density
Skewed Tree	Every node has at most one child	Worst-case (like linked list)

1.6 Step-by-Step (Node structure)

1. A node contains: data, left pointer, right pointer

2. Root is the topmost node

3. Left child is accessed via node->left

4. Right child is accessed via node->right

5. NULL pointer means no child exists

6. Tree is empty when root == NULL

1.7 Dry Run

Building tree: [1, 2, 3, 4, 5]

Step	Action	Tree State
1	Create root(1)	1
2	Insert 2 as left child of 1	1 -> left = 2
3	Insert 3 as right child of 1	1 -> right = 3
4	Insert 4 as left child of 2	2 -> left = 4
5	Insert 5 as right child of 2	2 -> right = 5

Result:

```

      1
     /\
    2  3
   /\
  4  5

```

1.8 C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

int main() {
    // Build tree: root(1) -> left=2, right=3 -> 2->left=4, 2->right=5
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    cout << "Root: " << root->val << "\n";
    cout << "Left child: " << root->left->val << "\n";
    cout << "Right child: " << root->right->val << "\n";

    // Output: Root: 1, Left child: 2, Right child: 3
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# Build tree
root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)

```

```
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)

print(f"Root: {root.val}")
print(f"Left child: {root.left.val}")
print(f"Right child: {root.right.val}")
```

1.9 Code Explanation

- **TreeNode** is self-referential — it has pointers to two other **TreeNode** objects
- Constructor initializes node value and sets children to NULL
- Building is done by linking nodes via **left** and **right** pointers
- Memory allocation uses **new** in C++ (must **delete** to avoid leaks — in interviews, not always required)

1.10 Complexity Analysis

Operation	Time	Space
Create node	O(1)	O(1)
Link children	O(1)	O(1)
Build n nodes	O(n)	O(n)

1.11 Common Patterns

- **Representing tree:** pointer-based, array-based (for complete trees)
- **Traversal:** recursive (simple) vs iterative (no stack overflow)
- **Construction:** from input, from array, from traversals

1.12 Edge Cases

- NULL / empty tree
- Single node (root is leaf)
- Skewed tree (left-heavy or right-heavy)
- Tree with duplicate values
- Very deep tree (recursion may stack overflow)

2. Tree Traversal

Note: All four traversals (Preorder, Inorder, Postorder, Level Order) share this section's structure. Individual sections below provide details specific to each.

Traversal means visiting **every node** in the tree exactly once, in a specific order.

Two categories:

1. **DFS-based:** Preorder, Inorder, Postorder (use stack/recursion)
 2. **BFS-based:** Level Order (uses queue)
-

3. Preorder Traversal

3.1 Overview

Preorder traversal visits nodes in the order: **Root** → **Left** → **Right**.

3.2 Intuition

You process the **current node first**, then recursively process the left subtree, then the right subtree. Think of reading a book: you read the chapter title first (root), then go through each section (left), then the next chapter (right).

3.3 When to Use It

- **Tree serialization** (reconstruct tree from traversal)
- **Creating a copy** of the tree
- **Prefix expression** evaluation (expression trees)
- Exploring root before children

3.4 When Not to Use It

- Need sorted order — use inorder
- Need children before parent — use postorder
- Need level-by-level processing — use level order

3.5 Core Concepts

- Visit order: root, left subtree, right subtree
- Root is visited **first**
- Useful for **top-down** processing

3.6 Step-by-Step Algorithm

1. If node is NULL, return
2. Process node (print / add to result)
3. Recursively traverse left subtree
4. Recursively traverse right subtree

3.7 Dry Run

Tree:



Step	Node	Visited	Action
1	1	1	Visit root 1
2	2	1,2	Go left
3	4	1,2,4	Go left of 2
4	NULL	-	Return from 4's left
5	NULL	-	Return from 4's right
6	5	1,2,4,5	Go right of 2
7	NULL	-	Return
8	3	1,2,4,5,3	Go right of 1
9	NULL	-	Return

Result: [1, 2, 4, 5, 3]

3.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// --- Recursive ---
void preorderRecursive(TreeNode* root, vector<int>& result) {
    if (!root) return;
    result.push_back(root->val);
    preorderRecursive(root->left, result);
    preorderRecursive(root->right, result);
}

// --- Iterative ---
vector<int> preorderIterative(TreeNode* root) {
    vector<int> result;
    if (!root) return result;
    stack<TreeNode*> st;
    st.push(root);
    while (!st.empty()) {
        TreeNode* node = st.top(); st.pop();
        result.push_back(node->val);
        // Push right first so left is processed first
        if (node->right) st.push(node->right);
        if (node->left) st.push(node->left);
    }
}
```

```

        return result;
    }

    int main() {
        TreeNode* root = new TreeNode(1);
        root->left = new TreeNode(2);
        root->right = new TreeNode(3);
        root->left->left = new TreeNode(4);
        root->left->right = new TreeNode(5);

        vector<int> res;
        preorderRecursive(root, res);
        for (int v : res) cout << v << " ";
        // Output: 1 2 4 5 3
        return 0;
    }

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def preorder_recursive(root, result=None):
    if result is None:
        result = []
    if not root:
        return result
    result.append(root.val)
    preorder_recursive(root.left, result)
    preorder_recursive(root.right, result)
    return result

def preorder_iterative(root):
    result = []
    if not root:
        return result
    stack = [root]
    while stack:
        node = stack.pop()
        result.append(node.val)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
    return result

# Example

```

```
root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
print(preorder_iterative(root)) # [1, 2, 4, 5, 3]
```

3.9 Code Explanation

- **Recursive:** Clean, mirrors definition directly. Stack frames hold state.
- **Iterative:** Uses explicit stack. Push right first then left so left is processed first (LIFO).
- Both visit each node exactly once.

3.10 Complexity Analysis

Aspect	Complexity
Time	$O(n)$ — each node visited once
Space (Recursive)	$O(h)$ — call stack, h = height
Space (Iterative)	$O(h)$ — explicit stack

Best case $h = O(\log n)$ (balanced), worst case $h = O(n)$ (skewed).

4. Inorder Traversal

4.1 Overview

Inorder traversal visits: **Left** → **Root** → **Right**.

4.2 Intuition

In a BST, inorder gives **sorted order**. Think of it as reading a book left-to-right: you go through the left side first (smaller things), then the middle, then the right side.

Analogy: Arranging books on a shelf by height — you pick the smallest first (far left), then work your way up.

4.3 When to Use It

- Get **sorted order** from a BST
- **Validate BST** (check if inorder is sorted)
- Convert BST to sorted array
- Find kth smallest/largest in BST

4.4 When Not to Use It

- Need root-first processing — use preorder
- Need children-before-parent — use postorder
- Tree is not a BST — sorted order not guaranteed

4.5 Core Concepts

- Left subtree completely processed before root

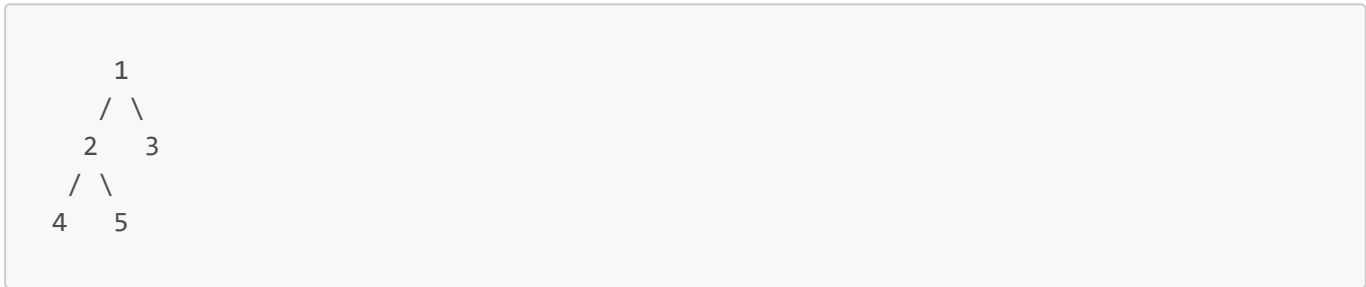
- Root processed before right subtree
- In BST: yields ascending order

4.6 Step-by-Step Algorithm

```
1. If node is NULL, return
2. Recursively traverse left subtree
3. Process node (print / add to result)
4. Recursively traverse right subtree
```

4.7 Dry Run

Tree:



Step	Node	Visited	Action
1	1	-	Going left
2	2	-	Going left
3	4	-	Going left (NULL)
4	4	4	Visit 4
5	4	4	Going right (NULL)
6	2	4,2	Visit 2
7	5	4,2	Going right
8	5	4,2,5	Visit 5
9	1	4,2,5,1	Visit 1
10	3	4,2,5,1	Going right
11	3	4,2,5,1,3	Visit 3

Result: [4, 2, 5, 1, 3]

4.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// --- Recursive ---
void inorderRecursive(TreeNode* root, vector<int>& result) {
    if (!root) return;
    inorderRecursive(root->left, result);
    result.push_back(root->val);
    inorderRecursive(root->right, result);
}

// --- Iterative ---
vector<int> inorderIterative(TreeNode* root) {
    vector<int> result;
    stack<TreeNode*> st;
    TreeNode* curr = root;
    while (curr || !st.empty()) {
        // Reach the leftmost node
        while (curr) {
            st.push(curr);
            curr = curr->left;
        }
        curr = st.top(); st.pop();
        result.push_back(curr->val);
        curr = curr->right; // Move to right subtree
    }
    return result;
}

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    vector<int> res;
    inorderRecursive(root, res);
    for (int v : res) cout << v << " ";
    // Output: 4 2 5 1 3
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):

```

```
        self.val = val
        self.left = left
        self.right = right

def inorder_recursive(root, result=None):
    if result is None:
        result = []
    if not root:
        return result
    inorder_recursive(root.left, result)
    result.append(root.val)
    inorder_recursive(root.right, result)
    return result

def inorder_iterative(root):
    result = []
    stack = []
    curr = root
    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        result.append(curr.val)
        curr = curr.right
    return result

root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
print(inorder_iterative(root)) # [4, 2, 5, 1, 3]
```

4.9 Code Explanation

- **Iterative:** Uses a stack. Go as far left as possible, pushing nodes. Pop and process, then go right. This simulates the recursion order exactly.
- **Key insight:** The `while (curr)` loop goes to extreme left, processing happens on pop, then we branch right.

4.10 Complexity Analysis

Aspect	Complexity
Time	$O(n)$
Space (Recursive)	$O(h)$
Space (Iterative)	$O(h)$

5. Postorder Traversal

5.1 Overview

Postorder traversal visits: **Left** → **Right** → **Root**.

5.2 Intuition

Children are processed **before** the parent. Think of deleting files: you must delete all files in a folder (children) before you can delete the folder (parent).

Analogy: A tournament bracket — you need results from both semi-finals (children) before you know the final match (root).

5.3 When to Use It

- **Delete a tree** (delete children before parent)
- **Expression tree evaluation** (evaluate children before operator at root)
- **Bottom-up DP** on trees
- **Tree destruction / memory cleanup**

5.4 When Not to Use It

- Need root-first — use preorder
- Need sorted order — use inorder

5.5 Core Concepts

- Left subtree processed → Right subtree processed → Root
- Parent depends on children's results
- **Bottom-up** processing

5.6 Step-by-Step Algorithm

1. If node is NULL, return
2. Recursively traverse left subtree
3. Recursively traverse right subtree
4. Process node (print / add to result)

5.7 Dry Run

Tree:



Step	Node	Visited	Action
1	1	-	Going left
2	2	-	Going left
3	4	-	Going left (NULL)
4	4	-	Going right (NULL)
5	4	4	Visit 4
6	5	4	Going right
7	5	-	Left (NULL)
8	5	-	Right (NULL)
9	5	4,5	Visit 5
10	2	4,5,2	Visit 2
11	3	4,5,2	Going right
12	3	-	Left (NULL)
13	3	-	Right (NULL)
14	3	4,5,2,3	Visit 3
15	1	4,5,2,3,1	Visit 1

Result: [4, 5, 2, 3, 1]

5.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// --- Recursive ---
void postorderRecursive(TreeNode* root, vector<int>& result) {
    if (!root) return;
    postorderRecursive(root->left, result);
    postorderRecursive(root->right, result);
    result.push_back(root->val);
}

// --- Iterative (two-stack method) ---
vector<int> postorderIterative(TreeNode* root) {
    vector<int> result;
```

```

if (!root) return result;
stack<TreeNode*> st1, st2;
st1.push(root);
while (!st1.empty()) {
    TreeNode* node = st1.top(); st1.pop();
    st2.push(node);
    if (node->left) st1.push(node->left);
    if (node->right) st1.push(node->right);
}
while (!st2.empty()) {
    result.push_back(st2.top()->val);
    st2.pop();
}
return result;
}

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    vector<int> res;
    postorderRecursive(root, res);
    for (int v : res) cout << v << " ";
    // Output: 4 5 2 3 1
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def postorder_recursive(root, result=None):
    if result is None:
        result = []
    if not root:
        return result
    postorder_recursive(root.left, result)
    postorder_recursive(root.right, result)
    result.append(root.val)
    return result

def postorder_iterative(root):
    result = []
    if not root:

```

```
        return result
    st1, st2 = [root], []
    while st1:
        node = st1.pop()
        st2.append(node)
        if node.left:
            st1.append(node.left)
        if node.right:
            st1.append(node.right)
    while st2:
        result.append(st2.pop().val)
    return result

root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
print(postorder_iterative(root)) # [4, 5, 2, 3, 1]
```

5.9 Code Explanation

- **Two-stack iterative:** First stack does modified preorder (root-right-left), second reverses it to get left-right-root.
- Alternative single-stack iterative is more complex but saves memory.

5.10 Complexity Analysis

Aspect	Complexity
Time	$O(n)$
Space (Recursive)	$O(h)$
Space (Iterative)	$O(n)$ in two-stack, $O(h)$ in single-stack

6. Level Order Traversal

6.1 Overview

Level order traversal visits nodes **level by level**, from left to right at each level. Also called **BFS** on a tree.

6.2 Intuition

Imagine printing a tree from top to bottom, left to right — exactly as it would appear visually.

Analogy: A waiter taking orders row by row in a theater. First row (left to right), then second row, then third...

6.3 When to Use It

- Find **shortest path** in unweighted tree
- **Level-based** processing (average per level, rightmost node per level)
- **Serialization** in BFS order
- Print tree in **visual format**
- Connect nodes at same level

6.4 When Not to Use It

- Need DFS-order (root-to-leaf) — use preorder
- Need entire depth-first processing — BFS queue can use O(n) memory

6.5 Core Concepts

- Uses a **queue** (FIFO)
- Process node, then enqueue its children
- Can separate levels by tracking queue size

6.6 Step-by-Step Algorithm

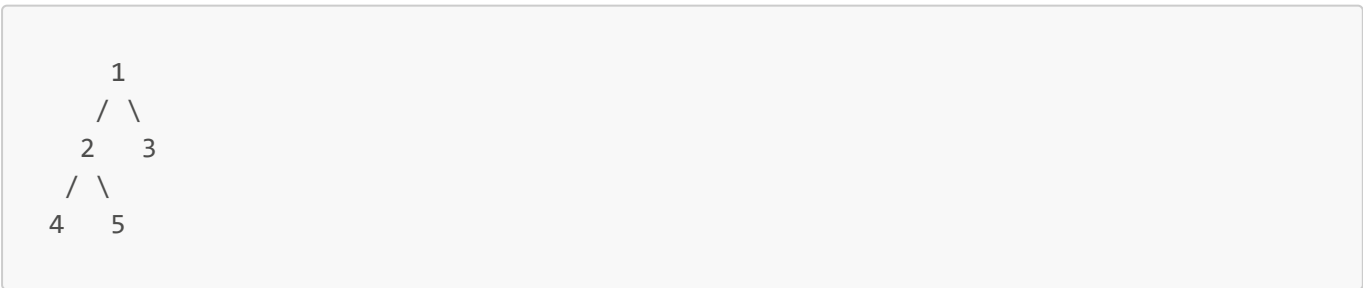
```
1. If root is NULL, return
2. Create queue, push root
3. While queue not empty:
  a. Dequeue node
  b. Process node
  c. Enqueue left child (if exists)
  d. Enqueue right child (if exists)
```

With level separation:

```
1. Create queue, push root
2. While queue not empty:
  a. size = queue.size()
  b. For i = 0 to size-1:
    - Dequeue, process
    - Enqueue children
  c. (End of level)
```

6.7 Dry Run

Tree:



Step	Queue (front→back)	Processed
1	[1]	-
2	[2,3]	1

Step	Queue (front→back)	Processed
3	[3,4,5]	1,2
4	[4,5]	1,2,3
5	[5]	1,2,3,4
6	[]	1,2,3,4,5

Level-by-level:

- Level 1: [1]
- Level 2: [2, 3]
- Level 3: [4, 5]

6.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// Basic level order
vector<int> levelOrder(TreeNode* root) {
    vector<int> result;
    if (!root) return result;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        TreeNode* node = q.front(); q.pop();
        result.push_back(node->val);
        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
    return result;
}

// Level order with level separation
vector<vector<int>> levelOrderWithLevels(TreeNode* root) {
    vector<vector<int>> result;
    if (!root) return result;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
```

```

        level.push_back(node->val);
        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
    result.push_back(level);
}
return result;
}

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    vector<int> res = levelOrder(root);
    for (int v : res) cout << v << " ";
    // Output: 1 2 3 4 5
    return 0;
}

```

Python Implementation

```

from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def level_order(root):
    if not root:
        return []
    result, q = [], deque([root])
    while q:
        node = q.popleft()
        result.append(node.val)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
    return result

def level_order_with_levels(root):
    if not root:
        return []
    result, q = [], deque([root])
    while q:
        level = []

```

```

        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
            result.append(level)
        return result

root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
print(level_order(root)) # [1, 2, 3, 4, 5]
print(level_order_with_levels(root)) # [[1], [2, 3], [4, 5]]

```

6.9 Code Explanation

- Queue maintains FIFO order, ensuring left-to-right processing at each level
- `size = q.size()` captures the current level's nodes before we process them
- The inner for-loop processes exactly one level

6.10 Complexity Analysis

Aspect	Complexity
Time	$O(n)$
Space	$O(w)$ — max width of tree (can be $O(n)$ for perfect tree's last level)

7. Height / Depth of Tree

7.1 Overview

Height of a node = number of edges on the longest path from that node to a leaf. **Height of tree** = height of root. **Depth** of a node = number of edges from root to that node.

7.2 Intuition

Think of a tree as a physical tree. The height is how tall the tree stands from its roots to its tallest leaf. Depth is how far down a particular branch point is from the top.

Analogy: A measuring tape. For height, measure from the root down to the furthest leaf. For depth of a node, measure from the root to that node.

7.3 When to Use It

- Determine if tree is **balanced**
- Complexity analysis of other tree algorithms
- **Diameter** calculation
- Level-based operations

7.4 When Not to Use It

- Only need number of nodes — use size
- Only need level of a specific node — can compute during traversal

7.5 Core Concepts

- **Height definition:** Some define as number of edges, some as number of nodes (off by 1)
- **Leaf height:** 0 (edges) or 1 (nodes)
- **Recursive formula:** $\text{height}(\text{node}) = 1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$

7.6 Step-by-Step Algorithm

1. If node is NULL, return 0 (or -1 depending on definition)
2. $\text{leftHeight} = \text{height}(\text{node} \rightarrow \text{left})$
3. $\text{rightHeight} = \text{height}(\text{node} \rightarrow \text{right})$
4. Return $1 + \max(\text{leftHeight}, \text{rightHeight})$

7.7 Dry Run

Tree:



Node	leftHeight	rightHeight	Height
4	0	0	1
5	0	0	1
2	1	1	2
3	0	0	1
1	2	1	3

Height of tree = 3 (node-based) or **2** (edge-based)

7.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
```

```

    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// Height (number of nodes on longest root-to-leaf path)
int height(TreeNode* root) {
    if (!root) return 0;
    return 1 + max(height(root->left), height(root->right));
}

// Height (number of edges on longest root-to-leaf path)
int heightEdges(TreeNode* root) {
    if (!root) return -1; // -1 so leaf becomes 0
    return 1 + max(heightEdges(root->left), heightEdges(root->right));
}

// Depth of a node (distance from root)
int depth(TreeNode* root, int target, int d = 0) {
    if (!root) return -1;
    if (root->val == target) return d;
    int left = depth(root->left, target, d + 1);
    if (left != -1) return left;
    return depth(root->right, target, d + 1);
}

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    cout << "Height (nodes): " << height(root) << "\n"; // 3
    cout << "Height (edges): " << heightEdges(root) << "\n"; // 2
    cout << "Depth of 4: " << depth(root, 4) << "\n"; // 2
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def height(root):
    if not root:
        return 0
    return 1 + max(height(root.left), height(root.right))

```

```
def height_edges(root):
    if not root:
        return -1
    return 1 + max(height_edges(root.left), height_edges(root.right))

def depth(root, target, d=0):
    if not root:
        return -1
    if root.val == target:
        return d
    left = depth(root.left, target, d + 1)
    if left != -1:
        return left
    return depth(root.right, target, d + 1)

root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
print(height(root)) # 3
```

7.9 Complexity Analysis

Aspect	Complexity
Time	$O(n)$
Space	$O(h)$ for recursion

8. Diameter of Tree

8.1 Overview

Diameter (or **width**) of a binary tree is the **longest path** between any two nodes. The path may or may not pass through the root. It's measured in **number of edges** (most common) or **number of nodes**.

8.2 Intuition

The diameter could be the longest path that goes through some node, combining its left subtree height and right subtree height. At each node, the longest path through it = `leftHeight + rightHeight`.

Analogy: The diameter of a circle is the longest distance across it. Similarly, the diameter of a tree is the longest distance between any two nodes.

8.3 When to Use It

- Find longest path in a tree
- Tree width problems
- Network diameter problems

8.4 When Not to Use It

- Need longest path from root to leaf — use height

- Tree is empty or has one node — diameter is 0

8.5 Core Concepts

- At any node, path through it = left height + right height
- Global maximum among all nodes = diameter
- Can be computed during height calculation ($O(n)$)

8.6 Step-by-Step Algorithm

```

1. Initialize global variable diameter = 0
2. For each node, compute height:
  a. leftHeight = height(root->left)
  b. rightHeight = height(root->right)
  c. diameter = max(diameter, leftHeight + rightHeight)
  d. Return 1 + max(leftHeight, rightHeight)
3. Final diameter after root's call is the answer

```

8.7 Dry Run

Tree:



Node	leftHeight	rightHeight	Candidate	max diameter
6	0	0	0	0
4	1	0	1	1
5	0	0	0	1
2	2	1	3	3
3	0	0	0	3
1	3	1	4	4

Diameter = 4 (path: 6→4→2→1→3, which is 4 edges)

8.8 C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    int diameterOfBinaryTree(TreeNode* root) {
        int diameter = 0;
        height(root, diameter);
        return diameter;
    }

private:
    int height(TreeNode* root, int& diameter) {
        if (!root) return 0;
        int leftH = height(root->left, diameter);
        int rightH = height(root->right, diameter);
        diameter = max(diameter, leftH + rightH);
        return 1 + max(leftH, rightH);
    }
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    root->left->left->left = new TreeNode(6);

    Solution sol;
    cout << "Diameter: " << sol.diameterOfBinaryTree(root) << "\n";
    // Output: 4
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:

```

```
def diameterOfBinaryTree(self, root):
    self.diameter = 0

    def height(node):
        if not node:
            return 0
        left = height(node.left)
        right = height(node.right)
        self.diameter = max(self.diameter, left + right)
        return 1 + max(left, right)

    height(root)
    return self.diameter

root = TreeNode(1, TreeNode(2, TreeNode(4, TreeNode(6)), TreeNode(5)),
                TreeNode(3))
print(Solution().diameterOfBinaryTree(root)) # 4
```

8.9 Code Explanation

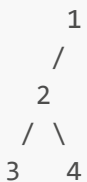
- We reuse the height function, but add a side effect: update `diameter` at each node
- `leftH + rightH` gives the longest path passing **through** current node
- The global maximum is tracked via reference

8.10 Complexity Analysis

Aspect	Complexity
Time	$O(n)$
Space	$O(h)$

8.11 Common Mistake

Assuming diameter always passes through root. **It doesn't.** Example:



Diameter = 2 (3→2→4), passes through node 2, not root 1.

9. Balanced Tree Check

9.1 Overview

A binary tree is **height-balanced** if, for every node, the height difference between its left and right subtrees is **at most 1**.

9.2 Intuition

A balanced tree is one where no branch is too much longer than another. Like a well-proportioned tree — all branches are roughly the same length.

Analogy: A balanced scale — the left and right sides are roughly equal in weight (height).

9.3 When to Use It

- Validate **AVL trees**
- Ensure $O(\log n)$ operations for BST
- Problems explicitly asking "is the tree balanced"

9.4 When Not to Use It

- Only need to check root's balance (not sufficient — every node must be checked)
- Using a tree that is inherently balanced (red-black, AVL) — they maintain it automatically

9.5 Core Concepts

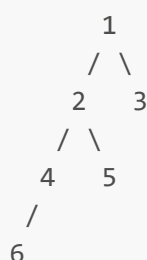
- **Balance condition:** $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$ for **all** nodes
- Can check in $O(n)$ by computing height and balance simultaneously
- Return **-1** as a sentinel for unbalanced, instead of using a boolean

9.6 Step-by-Step Algorithm

1. If node is NULL, return 0 (height = 0, balanced)
2. leftH = checkBalance(node->left)
3. If leftH == -1, return -1 (unbalanced)
4. rightH = checkBalance(node->right)
5. If rightH == -1, return -1 (unbalanced)
6. If $|\text{leftH} - \text{rightH}| > 1$, return -1
7. Return $1 + \max(\text{leftH}, \text{rightH})$

9.7 Dry Run

Tree:



Node	leftH	rightH	diff	Balanced?	Height
6	0	0	0	Yes	1
4	1	0	1	Yes	2
5	0	0	0	Yes	1
2	2	1	1	Yes	3
3	0	0	0	Yes	1
1	3	1	2	No	-1

9.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    bool isBalanced(TreeNode* root) {
        return checkHeight(root) != -1;
    }

private:
    // Returns height if balanced, -1 if unbalanced
    int checkHeight(TreeNode* root) {
        if (!root) return 0;

        int leftH = checkHeight(root->left);
        if (leftH == -1) return -1;

        int rightH = checkHeight(root->right);
        if (rightH == -1) return -1;

        if (abs(leftH - rightH) > 1) return -1;

        return 1 + max(leftH, rightH);
    }
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
```

```

root->left->right = new TreeNode(5);
root->left->left->left = new TreeNode(6);

Solution sol;
cout << "Is balanced: " << boolalpha << sol.isBalanced(root) << "\n";
// Output: false
return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def isBalanced(self, root):
        def check(node):
            if not node:
                return 0
            left = check(node.left)
            if left == -1:
                return -1
            right = check(node.right)
            if right == -1:
                return -1
            if abs(left - right) > 1:
                return -1
            return 1 + max(left, right)

        return check(root) != -1

root = TreeNode(1, TreeNode(2, TreeNode(4, TreeNode(6)), TreeNode(5)),
                TreeNode(3))
print(Solution().isBalanced(root)) # False

```

9.9 Complexity Analysis

Aspect	Complexity
Time	O(n)
Space	O(h)

10. Lowest Common Ancestor

10.1 Overview

The **Lowest Common Ancestor (LCA)** of two nodes p and q in a tree is the **deepest node** that is an ancestor of both p and q .

10.2 Intuition

Given two family members, find their closest common relative. The LCA is the node where the paths from root to p and root to q **diverge**.

Analogy: Imagine two branches of a river splitting from the main stream. The LCA is the point where the main stream splits.

10.3 When to Use It

- Find distance between two nodes
- Build path between two nodes
- Problems involving "common ancestor"
- Tree queries where you need to find common point

10.4 When Not to Use It

- Nodes don't exist in tree (must handle)
- Only one node present
- Tree is BST — use BST property (simple value comparison)
- Need multiple LCA queries — use binary lifting (preprocess)

10.5 Core Concepts

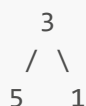
- If $root$ matches p or q , it's the LCA
- Recursively search left and right
- If both sides return non-null, $root$ is LCA
- If only one side returns non-null, that side's result is LCA

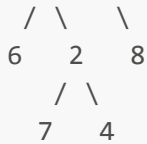
10.6 Step-by-Step Algorithm

1. If root is NULL, return NULL
2. If root == p or root == q, return root
3. left = LCA(root->left, p, q)
4. right = LCA(root->right, p, q)
5. If left != NULL and right != NULL, return root
6. Return left if left != NULL, else right

10.7 Dry Run

Tree:





Find LCA of 5 and 1:

- Root (3) is not 5 or 1
- left = LCA(5 subtree, 5, 1) → returns 5 (root of left subtree)
- right = LCA(1 subtree, 5, 1) → returns 1 (root of right subtree)
- Both non-null → return 3

Find LCA of 6 and 4:

- Root (3): left = ?
- Node 5: not 6 or 4
 - left = LCA(6, 6, 4) → returns 6
 - right = LCA(2, 6, 4) →
 - left = LCA(7, 6, 4) → NULL
 - right = LCA(4, 6, 4) → returns 4
 - Both non-null → returns 2
 - left=6, right=2 → both non-null → returns 5
- Returns 5

LCA of 6 and 4 = 5

10.8 C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (!root || root == p || root == q) return root;

        TreeNode* left = lowestCommonAncestor(root->left, p, q);
        TreeNode* right = lowestCommonAncestor(root->right, p, q);

        if (left && right) return root;
        return left ? left : right;
    }
};

```

```

int main() {
    TreeNode* root = new TreeNode(3);
    root->left = new TreeNode(5);
    root->right = new TreeNode(1);
    root->left->left = new TreeNode(6);
    root->left->right = new TreeNode(2);
    root->right->right = new TreeNode(8);
    root->left->right->left = new TreeNode(7);
    root->left->right->right = new TreeNode(4);

    Solution sol;
    TreeNode* lca = sol.lowestCommonAncestor(root, root->left->left, root->left->right->right);
    cout << "LCA of 6 and 4: " << lca->val << "\n"; // 5
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def lowestCommonAncestor(self, root, p, q):
        if not root or root == p or root == q:
            return root

        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)

        if left and right:
            return root
        return left if left else right

root = TreeNode(3, TreeNode(5, TreeNode(6), TreeNode(2, TreeNode(7),
TreeNode(4))), TreeNode(1, None, TreeNode(8)))
lca = Solution().lowestCommonAncestor(root, root.left.left, root.left.right.right)
print(lca.val) # 5

```

9.9 Complexity Analysis

Aspect	Complexity
Time	$O(n)$
Space	$O(h)$

11. Root-to-Leaf Paths

11.1 Overview

Find **all paths** from the root node to every leaf node in the binary tree.

11.2 Intuition

Start at root, walk down to each leaf, recording the path. When you hit a leaf, save the path. Then backtrack and try other branches.

Analogy: Exploring a cave system from entrance to all dead ends, marking your path each time.

11.3 When to Use It

- Print all root-to-leaf paths
- Path sum problems (modified)
- Find all possible sequences from root to leaf
- Tree serialization with all branches

11.4 When Not to Use It

- Only need one path — use DFS with early exit
- Need paths between any two nodes — use LCA-based approach

11.5 Core Concepts

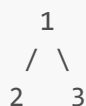
- **Backtracking:** add before recursive call, remove after
- Leaf: node with no children
- Path: sequence of node values from root to leaf

11.6 Step-by-Step Algorithm

1. Create empty vector for current path
2. Call DFS(root, path):
 - a. Add root->val to path
 - b. If root is leaf, add path to result
 - c. Else recurse for left and right
 - d. Remove root->val from path (backtrack)

11.7 Dry Run

Tree:



\
5

Call	Path	Action
dfs(1)	[1]	Add 1
dfs(2)	[1,2]	Add 2
dfs(NULL-left)	[1,2]	Skip
dfs(5)	[1,2,5]	Add 5, leaf → save [1,2,5]
Backtrack 5	[1,2]	Remove 5
Backtrack 2	[1]	Remove 2
dfs(3)	[1,3]	Add 3, leaf → save [1,3]
Backtrack 3	[1]	Remove 3

Result: `[[1,2,5], [1,3]]`

11.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    vector<string> binaryTreePaths(TreeNode* root) {
        vector<string> result;
        vector<int> path;
        dfs(root, path, result);
        return result;
    }

private:
    void dfs(TreeNode* root, vector<int>& path, vector<string>& result) {
        if (!root) return;

        path.push_back(root->val);

        // Leaf node
        if (!root->left && !root->right) {
            string s;
            for (int i = 0; i < path.size(); i++) {
```

```

        if (i > 0) s += "->";
        s += to_string(path[i]);
    }
    result.push_back(s);
} else {
    dfs(root->left, path, result);
    dfs(root->right, path, result);
}

path.pop_back(); // Backtrack
}

};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->right = new TreeNode(5);

    Solution sol;
    vector<string> paths = sol.binaryTreePaths(root);
    for (string p : paths) cout << p << "\n";
    // Output: 1->2->5, 1->3
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def binaryTreePaths(self, root):
        result = []

        def dfs(node, path):
            if not node:
                return
            path.append(str(node.val))
            if not node.left and not node.right:
                result.append("->".join(path))
            else:
                dfs(node.left, path)
                dfs(node.right, path)
            path.pop()

        dfs(root, [])
        return result

```

```
root = TreeNode(1, TreeNode(2, None, TreeNode(5)), TreeNode(3))
print(Solution().binaryTreePaths(root)) # ['1->2->5', '1->3']
```

11.9 Complexity Analysis

Aspect	Complexity
Time	$O(n^2)$ worst-case (copying paths of length $O(n)$ each) — or $O(n \times L)$ where L is number of leaf paths
Space	$O(n \times L)$ for storing all paths, $O(h)$ for recursion

12. Path Sum

12.1 Overview

Given a binary tree and a target sum, determine if there exists a **root-to-leaf path** such that the sum of all node values along the path equals the target.

12.2 Intuition

Walk from root to leaf, subtracting node values from target as you go. If at a leaf the remaining sum equals the leaf's value, we found a valid path.

Analogy: You have a budget (target) and each path has costs (node values). Find a route from root to leaf that exactly exhausts your budget.

12.3 When to Use It

- Path exists with given sum
- Path sum problems (variants: path sum II, path sum III)
- Subtree sum problems

12.4 When Not to Use It

- Path can start/end anywhere (not root-to-leaf) — use prefix sum + DFS
- Need all paths, not just existence — use path sum II variant

12.5 Core Concepts

- **Decrement target** as you go down
- At leaf, check `remaining == node->val`
- Early exit when found

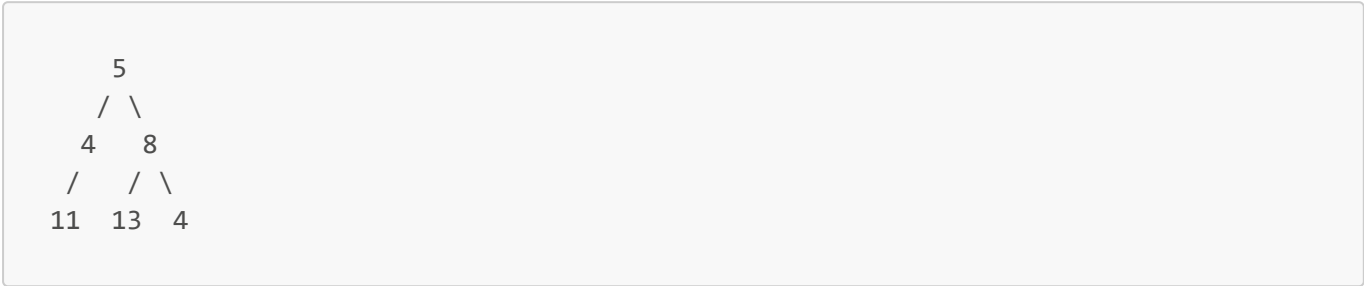
12.6 Step-by-Step Algorithm

1. If root is NULL, return false
2. `remainingSum = targetSum - root->val`

```
3. If root is leaf: return remainingSum == 0
4. Return hasPathSum(left, remainingSum) || hasPathSum(right, remainingSum)
```

12.7 Dry Run

Tree:



Target = 22

Node	Remaining	Action
5	22 - 5 = 17	Not leaf, recurse
4	17 - 4 = 13	Not leaf, recurse
11	13 - 11 = 2	Not leaf, recurse
7 (left of 11)	2 - 7 = -5	Leaf, not 0
2 (right of 11)	2 - 2 = 0	Leaf, 0 → found!

Result: true (path 5→4→11→2)

12.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    bool hasPathSum(TreeNode* root, int targetSum) {
        if (!root) return false;

        // Leaf check
        if (!root->left && !root->right) {
            return targetSum == root->val;
        }
    }
};
```

```

        int remaining = targetSum - root->val;
        return hasPathSum(root->left, remaining) ||
               hasPathSum(root->right, remaining);
    }
};

int main() {
    TreeNode* root = new TreeNode(5);
    root->left = new TreeNode(4);
    root->right = new TreeNode(8);
    root->left->left = new TreeNode(11);
    root->right->left = new TreeNode(13);
    root->right->right = new TreeNode(4);
    root->left->left->left = new TreeNode(7);
    root->left->left->right = new TreeNode(2);

    Solution sol;
    cout << sol.hasPathSum(root, 22) << "\n"; // 1 (true)
    cout << sol.hasPathSum(root, 26) << "\n"; // 0 (false)
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def hasPathSum(self, root, targetSum):
        if not root:
            return False
        if not root.left and not root.right:
            return targetSum == root.val
        remaining = targetSum - root.val
        return (self.hasPathSum(root.left, remaining) or
                self.hasPathSum(root.right, remaining))

root = TreeNode(5, TreeNode(4, TreeNode(11, TreeNode(7), TreeNode(2))),
                TreeNode(8, TreeNode(13), TreeNode(4)))
print(Solution().hasPathSum(root, 22)) # True

```

12.9 Complexity Analysis

Aspect	Complexity
Time	$O(n)$

Aspect	Complexity
Space	$O(h)$

13. Binary Search Tree

13.1 Overview

A **Binary Search Tree (BST)** is a binary tree with the **BST property**: For any node, all values in its left subtree are **less than** the node's value, and all values in its right subtree are **greater than** the node's value.

13.2 Intuition

Think of a BST as a **sorted array** cleverly organized as a tree. Each node splits the search space in half — go left if smaller, right if larger.

Analogy: A phone book. You open it in the middle. If the name you want comes before the current page, go to the left half; otherwise go to the right half. Each decision eliminates half the remaining space.

13.3 When to Use It

- **Dynamic set** with search, insert, delete (on average $O(\log n)$)
- **Ordered data** that needs to be traversed in sorted order
- **Floor/ceiling** queries
- **Range queries** (values between X and Y)
- **Kth smallest/largest** element

13.4 When Not to Use It

- Data fits in memory and is static — use sorted array + binary search ($O(\log n)$, no pointer overhead)
- Only need fast lookup without ordering — use hash table ($O(1)$)
- Random insert order leads to **skewed BST** — use balanced tree (AVL, Red-Black)
- Need only min/max — use heap or monotonic queue

13.5 Core Concepts

Concept	Explanation
BST Property	left < root < right (for all nodes)
Inorder	Gives sorted order
Successor	Next larger element (right subtree's leftmost)
Predecessor	Next smaller element (left subtree's rightmost)
Floor	Largest value $\leq$ target
Ceil	Smallest value $\geq$ target

13.6 Step-by-Step (Search)

1. If root is NULL, not found
2. If root->val == target, found
3. If target < root->val, search in left subtree
4. Else search in right subtree

13.7 C++ Implementation (Construction)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// Insert into BST (simple version without balancing)
TreeNode* insert(TreeNode* root, int val) {
    if (!root) return new TreeNode(val);
    if (val < root->val)
        root->left = insert(root->left, val);
    else if (val > root->val)
        root->right = insert(root->right, val);
    // If equal, we typically don't insert (or handle separately)
    return root;
}

// Build BST from array
TreeNode* buildBST(vector<int>& values) {
    TreeNode* root = NULL;
    for (int v : values) root = insert(root, v);
    return root;
}

void inorder(TreeNode* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->val << " ";
    inorder(root->right);
}

int main() {
    vector<int> values = {5, 3, 7, 2, 4, 6, 8};
    TreeNode* root = buildBST(values);
    inorder(root); // 2 3 4 5 6 7 8 (sorted)
    return 0;
}
```

Python Implementation

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def insert(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left = insert(root.left, val)
    elif val > root.val:
        root.right = insert(root.right, val)
    return root

def build_bst(values):
    root = None
    for v in values:
        root = insert(root, v)
    return root

def inorder(root):
    if not root:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)

values = [5, 3, 7, 2, 4, 6, 8]
root = build_bst(values)
print(inorder(root)) # [2, 3, 4, 5, 6, 7, 8]
```

13.8 Complexity Analysis (BST basics)

Operation	Average	Worst (skewed)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Inorder	$O(n)$	$O(n)$

14. Insert / Search / Delete in BST

14.1 Overview

Core BST operations: insert a value, search for a value, delete a value while maintaining BST property.

14.2 Intuition

- **Insert:** Walk down comparing values, find the correct NULL spot, create node

- **Search:** Walk down comparing, return when found or NULL
- **Delete:** Three cases → no child (just delete), one child (replace with child), two children (replace with inorder successor/predecessor)

14.3 When to Use It

- Dynamic set management
- Symbol tables
- Dictionary implementations
- Order statistic trees (with size field)

14.4 When Not to Use It

- Unbalanced insert order — use self-balancing BST
- Not needed if hash table suffices

14.5 Core Concepts

Delete — Three cases:

1. **Leaf:** Simply remove
2. **One child:** Replace node with its child
3. **Two children:** Find inorder successor (smallest in right subtree), copy its value, delete successor

14.6 Step-by-Step Algorithm (Delete)

1. Search for node with key
2. Case 1: Leaf → delete it
3. Case 2: One child → replace with child
4. Case 3: Two children →
 - a. Find inorder successor (right subtree's leftmost)
 - b. Copy successor's value to current node
 - c. Delete successor (which has at most one child, easy case)

14.7 Dry Run (Delete 3)

```

Initial:          Delete 3:
  5                5
 / \              / \
3   7            2   7
/ \              / \
2   4            1   4
/
1
  
```

- 3 has two children
- Successor of 3 = 4 (right's leftmost)

- Copy 4 to 3, delete original 4 (leaf → easy)

14.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// Search
TreeNode* search(TreeNode* root, int key) {
    if (!root || root->val == key) return root;
    if (key < root->val) return search(root->left, key);
    return search(root->right, key);
}

// Iterative search
TreeNode* searchIterative(TreeNode* root, int key) {
    while (root && root->val != key) {
        if (key < root->val) root = root->left;
        else root = root->right;
    }
    return root;
}

// Insert (returns new root)
TreeNode* insert(TreeNode* root, int val) {
    if (!root) return new TreeNode(val);
    if (val < root->val)
        root->left = insert(root->left, val);
    else if (val > root->val)
        root->right = insert(root->right, val);
    return root;
}

// Helper: find minimum in subtree
TreeNode* findMin(TreeNode* root) {
    while (root->left) root = root->left;
    return root;
}

// Delete
TreeNode* deleteNode(TreeNode* root, int key) {
    if (!root) return NULL;

    if (key < root->val) {
        root->left = deleteNode(root->left, key);
    } else if (key > root->val) {

```

```

        root->right = deleteNode(root->right, key);
    } else {
        // Found the node to delete
        // Case 1: Leaf
        if (!root->left && !root->right) {
            delete root;
            return NULL;
        }
        // Case 2: One child
        if (!root->left) {
            TreeNode* temp = root->right;
            delete root;
            return temp;
        }
        if (!root->right) {
            TreeNode* temp = root->left;
            delete root;
            return temp;
        }
        // Case 3: Two children
        TreeNode* successor = findMin(root->right);
        root->val = successor->val;
        root->right = deleteNode(root->right, successor->val);
    }
    return root;
}

void inorder(TreeNode* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->val << " ";
    inorder(root->right);
}

int main() {
    TreeNode* root = NULL;
    vector<int> vals = {5, 3, 7, 2, 4, 6, 8};
    for (int v : vals) root = insert(root, v);

    cout << "Before delete: "; inorder(root); cout << "\n";

    root = deleteNode(root, 3);
    cout << "After delete 3: "; inorder(root); cout << "\n";

    TreeNode* found = search(root, 7);
    cout << "Search 7: " << (found ? found->val : -1) << "\n";

    return 0;
}

```

Python Implementation

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def search(root, key):
    if not root or root.val == key:
        return root
    if key < root.val:
        return search(root.left, key)
    return search(root.right, key)

def insert(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left = insert(root.left, val)
    elif val > root.val:
        root.right = insert(root.right, val)
    return root

def find_min(root):
    while root.left:
        root = root.left
    return root

def delete_node(root, key):
    if not root:
        return None
    if key < root.val:
        root.left = delete_node(root.left, key)
    elif key > root.val:
        root.right = delete_node(root.right, key)
    else:
        if not root.left and not root.right:
            return None
        if not root.left:
            return root.right
        if not root.right:
            return root.left
        succ = find_min(root.right)
        root.val = succ.val
        root.right = delete_node(root.right, succ.val)
    return root

def inorder(root):
    if not root:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)

values = [5, 3, 7, 2, 4, 6, 8]
```

```
root = None
for v in values:
    root = insert(root, v)
print(inorder(root)) # [2, 3, 4, 5, 6, 7, 8]
root = delete_node(root, 3)
print(inorder(root)) # [2, 4, 5, 6, 7, 8]
```

14.9 Complexity Analysis

Operation	Average	Worst
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

15. Validate BST

15.1 Overview

Given a binary tree, determine if it is a **valid BST** — meaning every node obeys the BST property.

15.2 Intuition

Simply checking `left < root < right` at each node is **not enough**. You must propagate **range constraints** downward. Each node's value must lie within a valid range derived from its ancestors.

Analogy: A membership form — your age must be between 18 and 60. If you're in the "under 30" group, you still must be ≥ 18 . The constraints get tighter as you go deeper.

15.3 When to Use It

- Verify BST integrity
- Pre-requisite for BST applications
- Interview favorite

15.4 When Not to Use It

- Inorder traversal and check if sorted — simpler alternative
- Tree is guaranteed to be BST

15.5 Core Concepts

- Each node has a valid **range** (min, max)
- Root range: $(-\infty, +\infty)$ or (LONG_MIN, LONG_MAX)
- Left child's max = parent value
- Right child's min = parent value
- Use `long long` to avoid INT overflow issues

15.6 Step-by-Step Algorithm

1. Start with range (LONG_MIN, LONG_MAX)
2. If root is NULL, return true
3. If root->val <= min OR root->val >= max, return false
4. Left subtree must be valid with range (min, root->val)
5. Right subtree must be valid with range (root->val, max)
6. Return true if both are valid

15.7 Dry Run

Tree:



Check root 5: range $(-\infty, +\infty)$? Yes. Check 3: range $(-\infty, 5)$? 3 is in range. Yes. Check 2: range $(-\infty, 3)$? 2 is in range. Yes. Check 6: range $(3, 5)$? **6 > 5 → Invalid!**

Result: false (6 is in right of 3 but > 5 — violates BST property through ancestor)

15.8 C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    bool isValidBST(TreeNode* root) {
        return validate(root, LONG_MIN, LONG_MAX);
    }

private:
    bool validate(TreeNode* root, long long minVal, long long maxVal) {
        if (!root) return true;
        if (root->val <= minVal || root->val >= maxVal) return false;
        return validate(root->left, minVal, root->val) &&
            validate(root->right, root->val, maxVal);
    }
}

```

```
};

// Alternative: inorder approach
class Solution2 {
public:
    bool isValidBST(TreeNode* root) {
        TreeNode* prev = NULL;
        return inorder(root, prev);
    }

private:
    bool inorder(TreeNode* root, TreeNode*& prev) {
        if (!root) return true;
        if (!inorder(root->left, prev)) return false;
        if (prev && root->val <= prev->val) return false;
        prev = root;
        return inorder(root->right, prev);
    }
};

int main() {
    TreeNode* root = new TreeNode(5);
    root->left = new TreeNode(3);
    root->right = new TreeNode(7);
    root->left->left = new TreeNode(2);
    root->left->right = new TreeNode(6); // 6 > 5, invalid!

    Solution sol;
    cout << "Is valid: " << boolalpha << sol.isValidBST(root) << "\n";
    // Output: false
    return 0;
}
```

Python Implementation

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def isValidBST(self, root):
        def validate(node, lo, hi):
            if not node:
                return True
            if node.val <= lo or node.val >= hi:
                return False
            return (validate(node.left, lo, node.val) and
                    validate(node.right, node.val, hi))
```

```
        return validate(root, float('-inf'), float('inf'))

# Inorder approach
class Solution2:
    def isValidBST(self, root):
        self.prev = None

        def inorder(node):
            if not node:
                return True
            if not inorder(node.left):
                return False
            if self.prev is not None and node.val <= self.prev:
                return False
            self.prev = node.val
            return inorder(node.right)

        return inorder(root)
```

15.9 Complexity Analysis

Aspect	Complexity
Time	$O(n)$
Space	$O(h)$

16. Serialize / Deserialize Tree

16.1 Overview

Serialization converts a tree to a string (or array) so it can be stored/transmitted. **Deserialization** reconstructs the tree from that string.

16.2 Intuition

Flatten the tree into a string using a traversal (preorder or level order), marking NULL positions explicitly. Then rebuild using the same format.

Analogy: Taking apart a LEGO structure, writing down instructions (serialize), then rebuilding from those instructions (deserialize).

16.3 When to Use It

- Store tree in file/database
- Transmit over network
- Reconstruct after session
- Cache subtree computations

16.4 When Not to Use It

- Only need simple reconstruction from traversals (without NULLs) — requires both preorder + inorder
- Temporary in-memory use — just keep the tree

16.5 Core Concepts

- **NULL markers** are essential to uniquely reconstruct
- Preorder with NULL markers is most common
- Can use level order with NULL markers too
- Use a delimiter (, or)

16.6 Step-by-Step Algorithm (Preorder)

Serialize:

1. If root is NULL, append "#" and return
2. Append root->val
3. Recursively serialize left
4. Recursively serialize right

Deserialize:

1. Read next value from stream/queue
2. If it's "#", return NULL
3. Create node with this value
4. Recursively deserialize left
5. Recursively deserialize right
6. Return node

16.7 Dry Run

Tree:



Serialize: 1,2,4,#,#,5,#,#,3,#,#

Deserialize:

1. Read 1 → create node(1)
2. Read 2 → node(1)->left = node(2)
3. Read 4 → node(2)->left = node(4)
4. Read # → node(4)->left = NULL

5. Read # → node(4)->right = NULL
6. Read 5 → node(2)->right = node(5)
7. Read # → node(5)->left = NULL
8. Read # → node(5)->right = NULL
9. Read 3 → node(1)->right = node(3)
10. Read # → node(3)->left = NULL
11. Read # → node(3)->right = NULL

16.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Codec {
public:
    // Serialize
    string serialize(TreeNode* root) {
        ostringstream out;
        serializeHelper(root, out);
        return out.str();
    }

    // Deserialize
    TreeNode* deserialize(string data) {
        istringstream in(data);
        return deserializeHelper(in);
    }

private:
    void serializeHelper(TreeNode* root, ostringstream& out) {
        if (!root) {
            out << "# ";
            return;
        }
        out << root->val << " ";
        serializeHelper(root->left, out);
        serializeHelper(root->right, out);
    }

    TreeNode* deserializeHelper(istringstream& in) {
        string token;
        in >> token;
        if (token == "#") return NULL;

        TreeNode* root = new TreeNode(stoi(token));
```

```

        root->left = deserializeHelper(in);
        root->right = deserializeHelper(in);
        return root;
    }
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    Codec codec;
    string ser = codec.serialize(root);
    cout << "Serialized: " << ser << "\n";

    TreeNode* des = codec.deserialize(ser);
    cout << "Deserialized root: " << des->val << "\n";
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Codec:
    def serialize(self, root):
        def helper(node):
            if not node:
                return ["#"]
            return [str(node.val)] + helper(node.left) + helper(node.right)
        return " ".join(helper(root))

    def deserialize(self, data):
        tokens = iter(data.split())

        def helper():
            token = next(tokens)
            if token == "#":
                return None
            node = TreeNode(int(token))
            node.left = helper()
            node.right = helper()
            return node

        return helper()

```

```
root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
codec = Codec()
s = codec.serialize(root)
print(s)
new_root = codec.deserialize(s)
print(new_root.val) # 1
```

16.9 Complexity Analysis

Aspect	Complexity
Serialize Time	$O(n)$
Deserialize Time	$O(n)$
Space	$O(n)$ for string

17. Construct Tree from Traversals

17.1 Overview

Given two traversals (usually **inorder** + **preorder** or **inorder** + **postorder**), reconstruct the original binary tree.

17.2 Intuition

In preorder: first element is **root**. In inorder: elements left of root form left subtree, elements right of root form right subtree. Recursively build.

Analogy: You have two lists — one tells you "who is the captain" (preorder), another tells you "who is in left/right wing" (inorder).

17.3 When to Use It

- Given serialized traversals, reconstruct tree
- Understanding tree structure properties
- Interview question

17.4 When Not to Use It

- Need serialization with NULLs — simpler (just use preorder with #)
- Only one traversal given — cannot uniquely construct (unless it's a BST with range info)

17.5 Core Concepts

- **Pre+Inorder:** Preorder gives root, inorder gives split
- **Post+Inorder:** Postorder gives root (last element), same split logic
- **Pre+Post:** Ambiguous! Cannot uniquely determine tree (multiple trees possible)
- Use **hash map** to find inorder index quickly

17.6 Step-by-Step Algorithm (Pre+In)

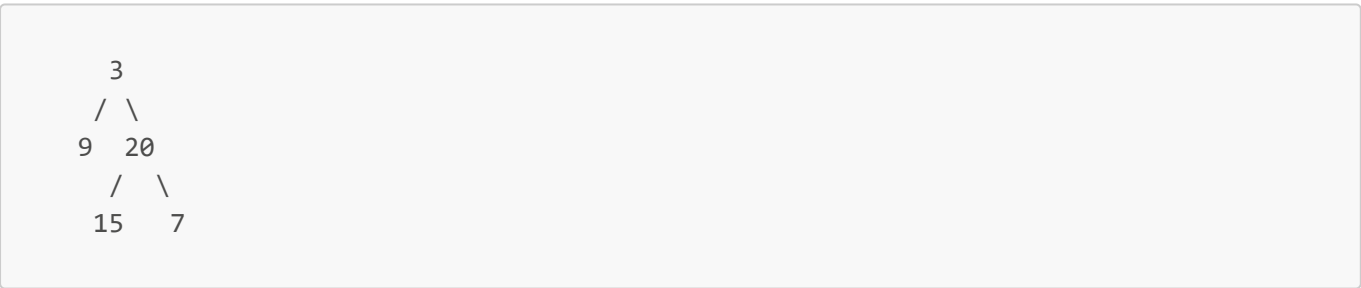
1. Preorder[0] is root
 2. Find root's index in inorder
 3. Left subtree size = rootIndex
 4. Build left: preorder[1...leftSize], inorder[0...rootIndex-1]
 5. Build right: preorder[1+leftSize...], inorder[rootIndex+1...]
 6. Recursively repeat

17.7 Dry Run

Preorder: [3, 9, 20, 15, 7] Inorder: [9, 3, 15, 20, 7]

Step	Preorder	Inorder	Root	Left Subtree	Right Subtree
1	[3,9,20,15,7]	[9,3,15,20,7]	3	[9] / [9]	[20,15,7] / [15,20,7]
2	[9]	[9]	9	NULL	NULL
3	[20,15,7]	[15,20,7]	20	[15] / [15]	[7] / [7]
4	[15]	[15]	15	NULL	NULL
5	[7]	[7]	7	NULL	NULL

Result:



17.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        unordered_map<int, int> inMap;
        for (int i = 0; i < inorder.size(); i++) {
```

```

        inMap[inorder[i]] = i;
    }
    return build(preorder, 0, preorder.size() - 1,
                inorder, 0, inorder.size() - 1, inMap);
}

private:
TreeNode* build(vector<int>& preorder, int preStart, int preEnd,
                vector<int>& inorder, int inStart, int inEnd,
                unordered_map<int, int>& inMap) {
    if (preStart > preEnd || inStart > inEnd) return NULL;

    int rootVal = preorder[preStart];
    TreeNode* root = new TreeNode(rootVal);

    int inRoot = inMap[rootVal];
    int leftSize = inRoot - inStart;

    root->left = build(preorder, preStart + 1, preStart + leftSize,
                      inorder, inStart, inRoot - 1, inMap);
    root->right = build(preorder, preStart + leftSize + 1, preEnd,
                       inorder, inRoot + 1, inEnd, inMap);

    return root;
}
};

int main() {
    vector<int> preorder = {3, 9, 20, 15, 7};
    vector<int> inorder = {9, 3, 15, 20, 7};

    Solution sol;
    TreeNode* root = sol.buildTree(preorder, inorder);

    // Verify by printing inorder
    vector<int> result;
    function<void(TreeNode*)> in = [&](TreeNode* r) {
        if (!r) return;
        in(r->left);
        result.push_back(r->val);
        in(r->right);
    };
    in(root);
    for (int v : result) cout << v << " ";
    cout << "\n"; // 9 3 15 20 7
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def buildTree(self, preorder, inorder):
        in_map = {v: i for i, v in enumerate(inorder)}

        def build(pre_start, pre_end, in_start, in_end):
            if pre_start > pre_end or in_start > in_end:
                return None

            root_val = preorder[pre_start]
            root = TreeNode(root_val)
            in_root = in_map[root_val]
            left_size = in_root - in_start

            root.left = build(pre_start + 1, pre_start + left_size,
                              in_start, in_root - 1)
            root.right = build(pre_start + left_size + 1, pre_end,
                               in_root + 1, in_end)

            return root

        return build(0, len(preorder) - 1, 0, len(inorder) - 1)

preorder = [3, 9, 20, 15, 7]
inorder = [9, 3, 15, 20, 7]
root = Solution().buildTree(preorder, inorder)

# Verify
def inorder_print(r):
    if not r:
        return []
    return inorder_print(r.left) + [r.val] + inorder_print(r.right)
print(inorder_print(root)) # [9, 3, 15, 20, 7]

```

17.9 Complexity Analysis

Aspect	Complexity
Time	$O(n)$ — each node processed once, hash map $O(1)$ lookup
Space	$O(n)$ for hash map, $O(h)$ for recursion

18. Vertical Order Traversal

18.1 Overview

Vertical order traversal visits nodes column by column. The root has column 0, left child gets column -1, right child gets column +1. At each column, nodes are ordered by row (top to bottom), and if same row, by value.

18.2 Intuition

Imagine projecting the tree onto a vertical axis. Each node has (column, row) coordinates. Group by column, sort by row then value.

Analogy: Columns in a spreadsheet. Root is in column 0. Move left → column decreases, move right → column increases.

18.3 When to Use It

- Print tree in vertical order
- Top/bottom view of binary tree
- Column-wise problems

18.4 When Not to Use It

- Only need level order — BFS is simpler
- Need symmetric check — mirror traversal is simpler

18.5 Core Concepts

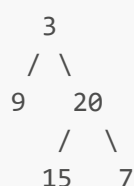
- **Column:** horizontal coordinate (root = 0, left = -1, right = +1)
- **Row:** vertical level (root = 0, child = parent_row + 1)
- **Sorting:** column asc → row asc → value asc

18.6 Step-by-Step Algorithm

1. Use DFS or BFS to assign (col, row) to each node
2. Store in map: col -> map<row, multiset<int>>
3. After traversal, sort columns
4. For each column, for each row (sorted), add values

18.7 Dry Run

Tree:



Coordinates:

- 3: (0, 0)
- 9: (-1, 1)
- 20: (1, 1)
- 15: (0, 2)
- 7: (2, 2)

By column:

- col -1: row 1 → [9]
- col 0: row 0 → [3], row 2 → [15] → [3, 15]
- col 1: row 1 → [20]
- col 2: row 2 → [7]

Result: [[9], [3,15], [20], [7]]

18.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    vector<vector<int>> verticalTraversal(TreeNode* root) {
        // map<col, map<row, multiset<int>>>
        map<int, map<int, multiset<int>>> nodes;

        // Queue for BFS: (node, col, row)
        queue<tuple<TreeNode*, int, int>> q;
        q.push({root, 0, 0});

        while (!q.empty()) {
            auto [node, col, row] = q.front(); q.pop();
            if (!node) continue;

            nodes[col][row].insert(node->val);
            q.push({node->left, col - 1, row + 1});
            q.push({node->right, col + 1, row + 1});
        }

        vector<vector<int>> result;
        for (auto& [col, rows] : nodes) {
            vector<int> colVals;
            for (auto& [row, vals] : rows) {
                colVals.insert(colVals.end(), vals.begin(), vals.end());
            }
            result.push_back(colVals);
        }
    }
};
```

```

    }
    return result;
}
};

int main() {
    TreeNode* root = new TreeNode(3);
    root->left = new TreeNode(9);
    root->right = new TreeNode(20);
    root->right->left = new TreeNode(15);
    root->right->right = new TreeNode(7);

    Solution sol;
    auto res = sol.verticalTraversal(root);
    for (auto& v : res) {
        for (int x : v) cout << x << " ";
        cout << "\n";
    }
    // Output: 9 \n 3 15 \n 20 \n 7
    return 0;
}

```

Python Implementation

```

from collections import deque, defaultdict

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def verticalTraversal(self, root):
        # col -> {row -> [values]}
        nodes = defaultdict(lambda: defaultdict(list))

        q = deque([(root, 0, 0)])
        while q:
            node, col, row = q.popleft()
            if not node:
                continue
            nodes[col][row].append(node.val)
            q.append((node.left, col - 1, row + 1))
            q.append((node.right, col + 1, row + 1))

        result = []
        for col in sorted(nodes.keys()):
            col_vals = []
            for row in sorted(nodes[col].keys()):
                col_vals.extend(sorted(nodes[col][row]))

```

```
        result.append(col_vals)
    return result

root = TreeNode(3, TreeNode(9), TreeNode(20, TreeNode(15), TreeNode(7)))
print(Solution().verticalTraversal(root)) # [[9], [3, 15], [20], [7]]
```

18.9 Complexity Analysis

Aspect	Complexity
Time	$O(n \log n)$ — due to sorting within columns
Space	$O(n)$

19. Boundary Traversal

19.1 Overview

Boundary traversal visits the boundary nodes of a binary tree in **anticlockwise direction**: left boundary (excluding leaf), leaf nodes (left to right), right boundary (excluding leaf, in reverse).

19.2 Intuition

Walk around the tree's perimeter. First go down the left side (excluding the bottom leaf, since leaf nodes will be handled separately), then collect all leaves left-to-right, then go up the right side in reverse (excluding the bottom leaf).

Analogy: Walking around the border of a country — first along the left coast, then the southern beaches, then back up the right coast.

19.3 When to Use It

- Print tree boundary
- Anticlockwise perimeter
- Edge detection in trees

19.4 When Not to Use It

- Need full traversal — use standard traversal
- Tree has only one node — handle as special case

19.5 Core Concepts

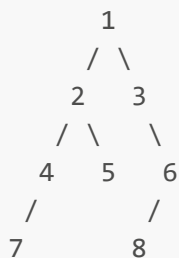
- **Left boundary:** root $\rightarrow$ leftmost path (excluding leaf)
- **Leaves:** all leaf nodes (left to right, via inorder or preorder)
- **Right boundary:** rightmost path (excluding leaf), collected in reverse
- **Avoid duplicates:** don't include root twice, don't include leaf in both left boundary and leaves

19.6 Step-by-Step Algorithm

1. Add root to result (if not leaf)
2. Traverse left boundary (excluding leaf):
 - a. If node has left child, go left; else go right
 - b. Stop when leaf is reached
3. Traverse leaves (left to right):
 - a. Add all leaf nodes
4. Traverse right boundary (excluding leaf):
 - a. If node has right child, go right; else go left
 - b. Collect in stack/vector, reverse at end
 - c. Stop when leaf is reached

19.7 Dry Run

Tree:



Boundary:

- Root: 1
- Left boundary: 2, 4 (stop before leaf 7 would be leaf)
- Leaves: 7, 5, 8
- Right boundary (reverse): 3 (6 is leaf, excluded)

Result: [1, 2, 4, 7, 5, 8, 3]

19.8 C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    vector<int> boundaryOfBinaryTree(TreeNode* root) {
        if (!root) return {};
        vector<int> result;
    }
};
  
```

```
        if (!isLeaf(root)) result.push_back(root->val);

        addLeftBoundary(root->left, result);
        addLeaves(root, result);
        addRightBoundary(root->right, result);

        return result;
    }

private:
    bool isLeaf(TreeNode* node) {
        return node && !node->left && !node->right;
    }

    void addLeftBoundary(TreeNode* node, vector<int>& result) {
        while (node && !isLeaf(node)) {
            result.push_back(node->val);
            if (node->left) node = node->left;
            else node = node->right;
        }
    }

    void addLeaves(TreeNode* node, vector<int>& result) {
        if (!node) return;
        if (isLeaf(node)) {
            result.push_back(node->val);
            return;
        }
        addLeaves(node->left, result);
        addLeaves(node->right, result);
    }

    void addRightBoundary(TreeNode* node, vector<int>& result) {
        vector<int> temp;
        while (node && !isLeaf(node)) {
            temp.push_back(node->val);
            if (node->right) node = node->right;
            else node = node->left;
        }
        // Add in reverse order
        for (int i = temp.size() - 1; i >= 0; i--) {
            result.push_back(temp[i]);
        }
    }
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    root->right->right = new TreeNode(6);
```

```

root->left->left->left = new TreeNode(7);
root->right->right->left = new TreeNode(8);

Solution sol;
auto res = sol.boundaryOfBinaryTree(root);
for (int v : res) cout << v << " ";
cout << "\n"; // 1 2 4 7 5 8 3
return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def boundaryOfBinaryTree(self, root):
        if not root:
            return []

        def is_leaf(node):
            return node and not node.left and not node.right

        result = []
        if not is_leaf(root):
            result.append(root.val)

        # Left boundary
        node = root.left
        while node and not is_leaf(node):
            result.append(node.val)
            node = node.left if node.left else node.right

        # Leaves
        def add_leaves(node):
            if not node:
                return
            if is_leaf(node):
                result.append(node.val)
                return
            add_leaves(node.left)
            add_leaves(node.right)
        add_leaves(root)

        # Right boundary (reversed)
        temp = []
        node = root.right
        while node and not is_leaf(node):

```

```
        temp.append(node.val)
        node = node.right if node.right else node.left
    result.extend(reversed(temp))

    return result

root = TreeNode(1, TreeNode(2, TreeNode(4, TreeNode(7)), TreeNode(5)), TreeNode(3,
None, TreeNode(6, TreeNode(8))))
print(Solution().boundaryOfBinaryTree(root)) # [1, 2, 4, 7, 5, 8, 3]
```

19.9 Complexity Analysis

Aspect	Complexity
Time	$O(n)$
Space	$O(n)$ for right boundary stack, $O(h)$ for recursion

20. Morris Traversal

20.1 Overview

Morris traversal performs **inorder** (or preorder) traversal using **$O(1)$ extra space** by creating temporary threads (pointers) from rightmost nodes in left subtree back to the current node.

20.2 Intuition

Morris traversal temporarily modifies the tree to create a **threaded binary tree**. When visiting a node, we find its **inorder predecessor** (rightmost node in left subtree) and link it back to current node. This lets us return to the correct node without a stack.

Analogy: Leaving breadcrumbs. Instead of a stack to remember where you came from, you create temporary paths that lead back, then remove them after use.

20.3 When to Use It

- Need **$O(1)$ space** traversal
- Cannot modify the tree (though Morris does modify temporarily — restore after)
- Large trees where recursion stack would overflow

20.4 When Not to Use It

- Tree is read-only (cannot modify, even temporarily)
- Simpler recursive/iterative is acceptable
- Need postorder — Morris postorder is complex

20.5 Core Concepts

- **Thread:** temporary link from predecessor to current node
- **Predecessor:** rightmost node in left subtree

- After processing left subtree, follow thread back and **remove it**
- Uses threading → detection → processing → unthreading

20.6 Step-by-Step Algorithm (Inorder)

```

1. curr = root
2. While curr != NULL:
  a. If curr->left == NULL:
    - Process curr
    - curr = curr->right
  b. Else:
    - Find predecessor: rightmost node in left subtree
    - If predecessor->right == NULL (thread not created):
      predecessor->right = curr // Create thread
      curr = curr->left
    - Else (thread exists, meaning left subtree done):
      predecessor->right = NULL // Remove thread
      Process curr
      curr = curr->right

```

20.7 Dry Run

Tree:



curr	left?	predecessor	Thread?	Action	Result
1	Yes	5 (rightmost of 2→5)	No	5→right=1, go left	-
2	Yes	4	No	4→right=2, go left	-
4	No	-	-	Process 4, go right	[4]
2 (via thread)	-	4 has right=2	Yes	Remove thread, process 2, go right	[4,2]
5	No	-	-	Process 5, go right (→1 via thread)	[4,2,5]
1 (via thread)	-	5 has right=1	Yes	Remove thread, process 1, go right	[4,2,5,1]
3	No	-	-	Process 3, go right (NULL)	[4,2,5,1,3]

Result: [4, 2, 5, 1, 3]

20.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// Morris Inorder Traversal (O(1) space)
vector<int> morrisInorder(TreeNode* root) {
    vector<int> result;
    TreeNode* curr = root;

    while (curr) {
        if (!curr->left) {
            // No left child: process and go right
            result.push_back(curr->val);
            curr = curr->right;
        } else {
            // Find inorder predecessor (rightmost in left subtree)
            TreeNode* pred = curr->left;
            while (pred->right && pred->right != curr) {
                pred = pred->right;
            }

            if (!pred->right) {
                // Create thread and go left
                pred->right = curr;
                curr = curr->left;
            } else {
                // Thread exists: left subtree done, remove thread
                pred->right = NULL;
                result.push_back(curr->val);
                curr = curr->right;
            }
        }
    }
    return result;
}

// Morris Preorder Traversal
vector<int> morrisPreorder(TreeNode* root) {
    vector<int> result;
    TreeNode* curr = root;

    while (curr) {
        if (!curr->left) {
            result.push_back(curr->val);
            curr = curr->right;
        }
    }
}
```

```

    } else {
        TreeNode* pred = curr->left;
        while (pred->right && pred->right != curr) {
            pred = pred->right;
        }

        if (!pred->right) {
            // Create thread, process current (preorder), go left
            pred->right = curr;
            result.push_back(curr->val);
            curr = curr->left;
        } else {
            // Thread exists: remove thread, go right
            pred->right = NULL;
            curr = curr->right;
        }
    }
}
return result;
}

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    auto inorder = morrisInorder(root);
    for (int v : inorder) cout << v << " ";
    cout << "\n"; // 4 2 5 1 3

    auto preorder = morrisPreorder(root);
    for (int v : preorder) cout << v << " ";
    cout << "\n"; // 1 2 4 5 3
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def morris_inorder(root):
    result = []
    curr = root
    while curr:
        if not curr.left:

```

```

        result.append(curr.val)
        curr = curr.right
    else:
        # Find predecessor
        pred = curr.left
        while pred.right and pred.right != curr:
            pred = pred.right

        if not pred.right:
            pred.right = curr
            curr = curr.left
        else:
            pred.right = None
            result.append(curr.val)
            curr = curr.right
    return result

def morris_preorder(root):
    result = []
    curr = root
    while curr:
        if not curr.left:
            result.append(curr.val)
            curr = curr.right
        else:
            pred = curr.left
            while pred.right and pred.right != curr:
                pred = pred.right

            if not pred.right:
                pred.right = curr
                result.append(curr.val)
                curr = curr.left
            else:
                pred.right = None
                curr = curr.right
    return result

root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
print(morris_inorder(root)) # [4, 2, 5, 1, 3]
print(morris_preorder(root)) # [1, 2, 4, 5, 3]

```

20.9 Complexity Analysis

Aspect	Complexity
Time	$O(n)$ — each edge traversed at most twice (finding predecessor + processing)
Space	$O(1)$ — no stack/queue (ignoring output array)

20.10 Common Mistake

Forgetting to **remove threads** — the tree structure is temporarily modified. Always restore it.

21. Binary Lifting for LCA

21.1 Overview

Binary lifting is a preprocessing technique that enables $O(\log n)$ queries for the **Lowest Common Ancestor (LCA)** of any two nodes, and related queries like **kth ancestor**.

21.2 Intuition

Precompute for each node its **2^j -th ancestor** for all j . Then to find LCA, "lift" the deeper node up to the same level as the shallower one, then lift both together until their ancestors match.

Analogy: Instead of climbing one step at a time from a node, you have a set of "super-steps": 1-step, 2-step, 4-step, 8-step... This lets you skip large chunks.

21.3 When to Use It

- Multiple LCA queries (after preprocessing)
- Find **kth ancestor** of a node
- Path queries (min, max, sum on path between nodes)
- Need $O(\log n)$ per query

21.4 When Not to Use It

- Single LCA query — use simple recursive $O(n)$ LCA
- Tree is small ($n < 1000$) — simple DFS is fine
- Cannot preprocess (one-off query on static tree)

21.5 Core Concepts

- **up[node][j]**: 2^j -th ancestor of node (up[node][0] = parent, up[node][1] = grandparent, etc.)
- **LOG**: $\text{ceil}(\log_2(N)) + 1$, typically 20 for 10^6
- **Depth**: distance from root
- **Lifting**: jump 2^j steps when the j -th bit of the required jump is set

21.6 Step-by-Step Algorithm

Preprocessing:

1. DFS from root, compute depth and parent (up[node][0])
2. For $j = 1$ to $\text{LOG}-1$:

$$\text{up}[\text{node}][j] = \text{up}[\text{up}[\text{node}][j-1]][j-1]$$

LCA(u, v):

1. If $\text{depth}[u] < \text{depth}[v]$, swap
2. Lift u up so $\text{depth}[u] == \text{depth}[v]$:

For $j = \text{LOG}-1$ down to 0:

If $\text{depth}[u] - (1 \ll j) \geq \text{depth}[v]$:

$$u = \text{up}[u][j]$$

```

3. If  $u == v$ , return  $u$ 
4. For  $j = \text{LOG}-1$  down to  $0$ :
    If  $\text{up}[u][j] \neq \text{up}[v][j]$ :
         $u = \text{up}[u][j]; v = \text{up}[v][j]$ 
5. Return  $\text{up}[u][0]$  (parent of  $u/v$ )

```

Kth ancestor(u, k):

```

1. For  $j = 0$  to  $\text{LOG}-1$ :
    if  $(k \gg j) \ \& \ 1$ :
         $u = \text{up}[u][j]$ 
2. Return  $u$ 

```

21.7 Dry Run

Tree:



Preprocessing ($\text{LOG} = 3$):

Node	$\text{up}[\text{node}][0]$	$\text{up}[\text{node}][1]$	$\text{up}[\text{node}][2]$	depth
1	1 (or 0)	1	1	0
2	1	1 ($\text{up}[1][0] = 1$)	1	1
3	1	1	1	1
4	2	1	1	2
5	2	1	1	2

LCA(4, 5):

- depth 4=2, depth 5=2 (equal)
- $j=2$: $\text{up}[4][2] = 1, \text{up}[5][2] = 1 \rightarrow$ same, skip
- $j=1$: $\text{up}[4][1] = 1, \text{up}[5][1] = 1 \rightarrow$ same, skip
- $j=0$: $\text{up}[4][0] = 2, \text{up}[5][0] = 2 \rightarrow$ same, skip
- Return $\text{up}[4][0] = 2$

LCA(4, 3):

- depth 4=2 > depth 3=1
- lift 4: $j=1$ (2 steps): depth 4 becomes 0 $\rightarrow$ overshoot
- $j=0$ (1 step): depth 4 becomes 1
- Now $u=2$ (depth 1), $v=3$ (depth 1)

- $j=1$: both $up=1$, skip
- $j=0$: $up[2][0]=1$, $up[3][0]=1 \rightarrow$ same, skip
- Return $up[2][0] = 1$

21.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class BinaryLifting {
    int n, LOG;
    vector<vector<int>> adj;
    vector<int> depth;
    vector<vector<int>> up; // up[node][j] = 2^j-th ancestor

public:
    BinaryLifting(int n, vector<vector<int>>& adj, int root = 1)
        : n(n), adj(adj) {
        LOG = ceil(log2(n)) + 1;
        depth.assign(n + 1, 0);
        up.assign(n + 1, vector<int>(LOG, 0));
        dfs(root, 0);
    }

    void dfs(int node, int parent) {
        up[node][0] = parent;
        for (int j = 1; j < LOG; j++) {
            up[node][j] = up[up[node][j-1]][j-1];
        }
        for (int child : adj[node]) {
            if (child != parent) {
                depth[child] = depth[node] + 1;
                dfs(child, node);
            }
        }
    }

    int lca(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);

        // Lift u to same depth as v
        int diff = depth[u] - depth[v];
        for (int j = 0; j < LOG; j++) {
            if (diff & (1 << j)) {
                u = up[u][j];
            }
        }

        if (u == v) return u;

        // Lift both together
        for (int j = LOG - 1; j >= 0; j--) {
```

```

        if (up[u][j] != up[v][j]) {
            u = up[u][j];
            v = up[v][j];
        }
    }

    return up[u][0];
}

int kthAncestor(int u, int k) {
    for (int j = 0; j < LOG; j++) {
        if (k & (1 << j)) {
            u = up[u][j];
            if (u == 0) break; // beyond root
        }
    }
    return u;
}

int distance(int u, int v) {
    int l = lca(u, v);
    return depth[u] + depth[v] - 2 * depth[l];
}
};

int main() {
    int n = 5;
    vector<vector<int>> adj(n + 1);
    adj[1] = {2, 3};
    adj[2] = {1, 4, 5};
    adj[3] = {1};
    adj[4] = {2};
    adj[5] = {2};

    BinaryLifting bl(n, adj, 1);

    cout << "LCA(4,5): " << bl.lca(4, 5) << "\n"; // 2
    cout << "LCA(4,3): " << bl.lca(4, 3) << "\n"; // 1
    cout << "2nd ancestor of 4: " << bl.kthAncestor(4, 2) << "\n"; // 1
    cout << "Distance(4,3): " << bl.distance(4, 3) << "\n"; // 3
    return 0;
}

```

Python Implementation

```

import math

class BinaryLifting:
    def __init__(self, n, adj, root=1):
        self.n = n
        self.adj = adj

```

```

self.LOG = math.ceil(math.log2(n)) + 1
self.depth = [0] * (n + 1)
self.up = [[0] * self.LOG for _ in range(n + 1)]
self.dfs(root, 0)

def dfs(self, node, parent):
    self.up[node][0] = parent
    for j in range(1, self.LOG):
        self.up[node][j] = self.up[self.up[node][j-1]][j-1]
    for child in self.adj[node]:
        if child != parent:
            self.depth[child] = self.depth[node] + 1
            self.dfs(child, node)

def lca(self, u, v):
    if self.depth[u] < self.depth[v]:
        u, v = v, u

    diff = self.depth[u] - self.depth[v]
    for j in range(self.LOG):
        if diff & (1 << j):
            u = self.up[u][j]

    if u == v:
        return u

    for j in range(self.LOG - 1, -1, -1):
        if self.up[u][j] != self.up[v][j]:
            u = self.up[u][j]
            v = self.up[v][j]

    return self.up[u][0]

def kth_ancestor(self, u, k):
    for j in range(self.LOG):
        if k & (1 << j):
            u = self.up[u][j]
            if u == 0:
                break
    return u

def distance(self, u, v):
    l = self.lca(u, v)
    return self.depth[u] + self.depth[v] - 2 * self.depth[l]

# Example
n = 5
adj = [[] for _ in range(n + 1)]
adj[1] = [2, 3]
adj[2] = [1, 4, 5]
adj[3] = [1]
adj[4] = [2]
adj[5] = [2]

```

```
bl = BinaryLifting(n, adj, 1)
print(bl.lca(4, 5)) # 2
print(bl.lca(4, 3)) # 1
print(bl.kth_ancestor(4, 2)) # 1
print(bl.distance(4, 3)) # 3
```

21.9 Complexity Analysis

Aspect	Complexity
Preprocessing Time	$O(n \log n)$
Per Query (LCA)	$O(\log n)$
Per Query (kth ancestor)	$O(\log n)$
Space	$O(n \log n)$

22. Tree DP

22.1 Overview

Tree DP (Dynamic Programming on Trees) refers to solving optimization problems by computing DP values for each node based on its children's DP values — typically bottom-up (postorder traversal).

22.2 Intuition

A tree has a recursive structure. If we know the answer for all children of a node, we can compute the answer for the node itself. This is a **bottom-up** DP.

Analogy: A company with departments. To know the total profit, each department head collects data from their teams, processes it, and reports upward.

22.3 When to Use It

- **Maximum independent set:** Choose nodes with no adjacent nodes
- **Tree diameter:** Longest path (can be done with DP)
- **Subtree sums/sizes**
- **Tree coloring problems** (e.g., vertex cover)
- **Distance-related DP** (sum of distances in tree)
- **Rerooting DP** (compute DP for all roots)

22.4 When Not to Use It

- Linear chain structure — simpler DP works
- No overlapping subproblems — greedy may work
- Tree is very large and recursion depth is a concern — use iterative postorder

22.5 Core Concepts

- **State definition:** `dp[node]` = answer for subtree rooted at node

- **Transition:** combine children's DP values
- **Base case:** leaf node (no children)
- **Rerooting:** compute DP for root, then propagate to children to get DP for all nodes as roots

22.6 Step-by-Step Algorithm (Maximum Independent Set)

Problem: Choose max set of nodes such that no two are adjacent.

State:

$dp[node][0]$ = max independent set in subtree of node, node NOT taken

$dp[node][1]$ = max independent set in subtree of node, node TAKEN

Transition:

$dp[node][0] = \sum(\max(dp[child][0], dp[child][1]) \text{ for all children})$

$dp[node][1] = 1 + \sum(dp[child][0] \text{ for all children})$

Base: leaf's $dp = \{0, 1\}$

Answer: $\max(dp[root][0], dp[root][1])$

22.7 Dry Run

Tree:



Node	$dp[node][0]$	$dp[node][1]$	Calculation
4	0	1	Leaf
5	0	1	Leaf
2	$\max(0,1) + \max(0,1) = 2$	$1 + 0 + 0 = 1$	$dp[2][0] = dp[4][0] + dp[4][1]$ etc.
3	0	1	Leaf
1	$\max(2,1) + \max(0,1) = 3$	$1 + 2 + 0 = 3$	$dp[1][0] = \max \text{ of children}$

Answer: $\max(3, 3) = 3$

22.8 C++ Implementation (Maximum Independent Set / House Robber III)

```
#include <bits/stdc++.h>
using namespace std;
```

```

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// House Robber III (Maximum independent set on binary tree)
class Solution {
public:
    int rob(TreeNode* root) {
        auto [notTaken, taken] = dfs(root);
        return max(notTaken, taken);
    }

private:
    // Returns {max_gain_without_robbing_this, max_gain_with_robbing_this}
    pair<int, int> dfs(TreeNode* node) {
        if (!node) return {0, 0};

        auto [l_not, l_taken] = dfs(node->left);
        auto [r_not, r_taken] = dfs(node->right);

        // If we don't rob this node, we can rob or not rob children
        int notTaken = max(l_not, l_taken) + max(r_not, r_taken);

        // If we rob this node, we must not rob children
        int taken = node->val + l_not + r_not;

        return {notTaken, taken};
    }
};

// Tree DP: sum of distances in tree (general tree, rerooting)
class TreeDistSum {
    int n;
    vector<vector<int>>> adj;
    vector<long long> dp; // dp[u] = sum of distances from u to all nodes
    vector<int> sz; // subtree sizes

public:
    TreeDistSum(int n, vector<vector<int>>>& adj) : n(n), adj(adj) {
        dp.assign(n, 0);
        sz.assign(n, 0);
    }

    vector<long long> solve() {
        // First DFS: compute dp[root] and subtree sizes
        dfs1(0, -1);
        // Second DFS: reroot and compute dp for all nodes
        dfs2(0, -1);
        return dp;
    }

private:

```

```

void dfs1(int u, int p) {
    sz[u] = 1;
    for (int v : adj[u]) {
        if (v == p) continue;
        dfs1(v, u);
        sz[u] += sz[v];
        dp[0] += dp[v] + sz[v]; // contribution of subtree v to root
    }
}

void dfs2(int u, int p) {
    for (int v : adj[u]) {
        if (v == p) continue;
        // Reroot from u to v
        dp[v] = dp[u] + (n - sz[v]) - sz[v];
        dfs2(v, u);
    }
}

};

int main() {
    // House Robber example
    TreeNode* root = new TreeNode(3);
    root->left = new TreeNode(4);
    root->right = new TreeNode(5);
    root->left->left = new TreeNode(1);
    root->left->right = new TreeNode(3);
    root->right->right = new TreeNode(1);

    Solution sol;
    cout << "Max rob: " << sol.rob(root) << "\n"; // 9

    // Sum of distances example
    int n = 6;
    vector<vector<int>> adj(n);
    adj[0] = {1, 2};
    adj[1] = {0, 3, 4};
    adj[2] = {0, 5};
    adj[3] = {1};
    adj[4] = {1};
    adj[5] = {2};

    TreeDistSum tds(n, adj);
    auto res = tds.solve();
    for (long long d : res) cout << d << " ";
    cout << "\n"; // distances from each node to all others
    return 0;
}

```

Python Implementation

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# House Robber III
class Solution:
    def rob(self, root):
        def dfs(node):
            if not node:
                return (0, 0)
            l_not, l_taken = dfs(node.left)
            r_not, r_taken = dfs(node.right)

            not_taken = max(l_not, l_taken) + max(r_not, r_taken)
            taken = node.val + l_not + r_not

            return (not_taken, taken)

        return max(dfs(root))

# Sum of distances using rerooting
class TreeDistSum:
    def __init__(self, n, adj):
        self.n = n
        self.adj = adj
        self.dp = [0] * n
        self.sz = [0] * n

    def solve(self):
        self.dfs1(0, -1)
        self.dfs2(0, -1)
        return self.dp

    def dfs1(self, u, p):
        self.sz[u] = 1
        for v in self.adj[u]:
            if v == p:
                continue
            self.dfs1(v, u)
            self.sz[u] += self.sz[v]
            self.dp[0] += self.dp[v] + self.sz[v]

    def dfs2(self, u, p):
        for v in self.adj[u]:
            if v == p:
                continue
            self.dp[v] = self.dp[u] + (self.n - self.sz[v]) - self.sz[v]
            self.dfs2(v, u)

root = TreeNode(3, TreeNode(4, TreeNode(1), TreeNode(3)), TreeNode(5, None,

```

```
TreeNode(1)))
print(Solution().rob(root)) # 9
```

22.9 Complexity Analysis

Aspect	Complexity
Time	$O(n)$ — each node visited once (or twice for rerooting)
Space	$O(n)$ for DP arrays, $O(h)$ for recursion

22.10 Common Patterns

- **Subtree sum/size:** `dp[u] = val[u] + sum(dp[child])`
- **Diameter:** `diameter = max(diameter, height[child1] + height[child2])` across all nodes
- **Max independent set:** 2-state DP (take/not-take)
- **Rerooting:** Compute for root, then propagate to children

23. Heavy-Light Decomposition

23.1 Overview

Heavy-Light Decomposition (HLD) splits a tree into **chains** (paths) such that any root-to-leaf path goes through at most $O(\log n)$ chains. This allows **path queries** (min, max, sum between two nodes) to be answered in $O(\log^2 n)$ using a segment tree.

23.2 Intuition

Break the tree into "heavy" paths. The heavy child is the one with the largest subtree size. Light edges (connecting different chains) are at most $O(\log n)$ on any root-to-leaf path. Path queries become queries on $O(\log n)$ contiguous segments.

Analogy: A highway system with expressways (heavy chains) and local roads (light edges). You can travel most of the distance on expressways, switching only a few times.

23.3 When to Use It

- **Path queries** between two nodes (sum, max, min, XOR)
- **Path updates** (add value to all nodes on path)
- **Subtree queries** (using Euler tour + segment tree)
- Problems requiring both path and subtree operations

23.4 When Not to Use It

- Only subtree queries — Euler tour + segment tree is simpler
- Tree is small — brute force is fine
- Queries are static (no updates) — binary lifting + prefix sums may work
- Only LCA needed — binary lifting is simpler

23.5 Core Concepts

Concept	Definition
Heavy child	Child with largest subtree size
Light child	Any child that is not heavy
Heavy edge	Edge from node to its heavy child
Chain	Sequence of nodes connected by heavy edges
Head	Topmost node of a chain
Position	Index in segment tree array (base array)

23.6 Step-by-Step Algorithm

Preprocessing:

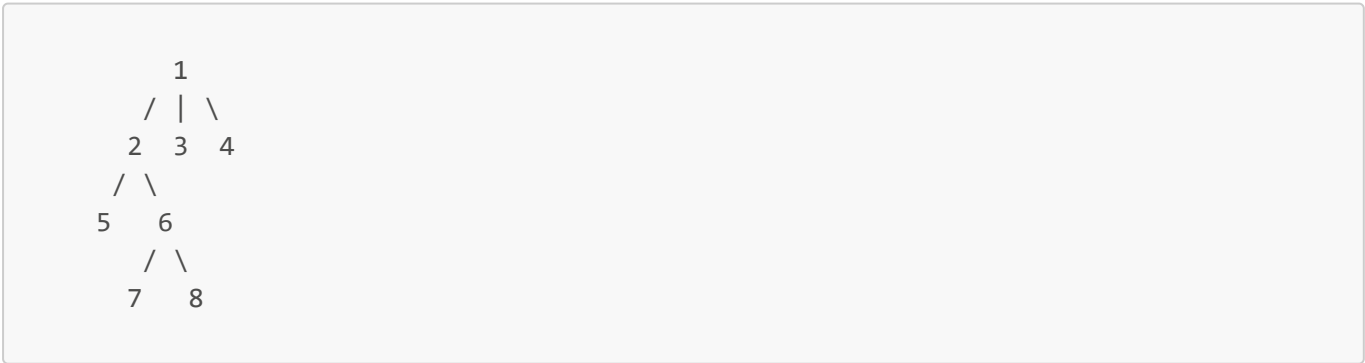
1. First DFS: compute subtree sizes, parent, depth, heavy child
2. Second DFS: assign chain heads, positions (base array index)

Path Query (u, v):

1. While head[u] != head[v]:
 - a. If depth[head[u]] < depth[head[v]], swap
 - b. Query segment tree on [pos[head[u]], pos[u]]
 - c. u = parent[head[u]]
2. If depth[u] > depth[v], swap
3. Query segment tree on [pos[u], pos[v]]
4. Return combined result

23.7 Dry Run

Tree:



Subtree sizes: 8(1), 4(2, with 5,6,7,8), 1(3), 1(4), 1(5), 3(6, with 7,8), 1(7), 1(8)

Heavy children:

- 1 → 2 (size 4 vs 1 vs 1)
- 2 → 6 (size 3 vs 1)
- 6 → 8 (size 1 == 1, tie-break any)

Chains:

- Chain 1: 1-2-6-8 (head=1)
- Chain 2: 3 (head=3)
- Chain 3: 4 (head=4)
- Chain 4: 5 (head=5)
- Chain 5: 7 (head=7)

Positions (base array): [1, 2, 6, 8, 5, 3, 4, 7]

Path 5 → 3:

- head[5]=5, head[3]=3. depth[head[5]]=depth[1]? No, depth[5]=2, depth[3]=1
- depth[head[5]]=depth[1]=0 < depth[head[3]]=depth[3]=1? No. So swap.
- head[3]=3, head[5]=5. depth[head[3]]=depth[3]=1 < depth[head[5]]=depth[1]=0? No.
- Query [pos[head[3]], pos[3]] = segment for node 3
- u = parent[head[3]] = parent[3] = 1
- Now head[1]=1, head[5]=5. depth[head[1]]=0 < depth[head[5]]=0? No.
- Query [pos[head[5]], pos[5]] = segment for node 5
- u = parent[head[5]] = parent[5] = 2
- Now head[1]=1, head[2]=1. head[1]==head[2].
- depth[1]=0 < depth[2]=1. Query [pos[1], pos[2]]
- Path: 3 → 1 → 2 → 5

23.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class HLD {
    int n;
    vector<vector<int>>> adj;
    vector<int> parent, depth, heavy, head, pos, sz, arr;
    vector<int> segtree; // segment tree for path queries
    int curPos;

public:
    HLD(int n, vector<vector<int>>> adj) : n(n), adj(adj) {
        parent.assign(n, -1);
        depth.assign(n, 0);
        heavy.assign(n, -1);
        head.assign(n, 0);
        pos.assign(n, 0);
        sz.assign(n, 0);
        arr.assign(n, 0);
        curPos = 0;

        dfs(0, -1);
        decompose(0, 0);

        segtree.assign(4 * n, 0);
    }
};
```

```

}

void setValue(int node, int val) {
    arr[pos[node]] = val;
}

void buildSegmentTree() {
    build(1, 0, n - 1);
}

// Path query (sum, max, min, etc.)
int pathQuery(int u, int v) {
    int result = 0; // For sum
    while (head[u] != head[v]) {
        if (depth[head[u]] < depth[head[v]]) swap(u, v);
        result += query(1, 0, n - 1, pos[head[u]], pos[u]);
        u = parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    result += query(1, 0, n - 1, pos[u], pos[v]);
    return result;
}

// Path update (add val to all nodes on path)
void pathUpdate(int u, int v, int val) {
    while (head[u] != head[v]) {
        if (depth[head[u]] < depth[head[v]]) swap(u, v);
        update(1, 0, n - 1, pos[head[u]], pos[u], val);
        u = parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    update(1, 0, n - 1, pos[u], pos[v], val);
}

private:
int dfs(int u, int p) {
    parent[u] = p;
    sz[u] = 1;
    int maxSz = 0;
    for (int v : adj[u]) {
        if (v == p) continue;
        depth[v] = depth[u] + 1;
        int subSz = dfs(v, u);
        sz[u] += subSz;
        if (subSz > maxSz) {
            maxSz = subSz;
            heavy[u] = v;
        }
    }
    return sz[u];
}

void decompose(int u, int h) {
    head[u] = h;

```

```

    pos[u] = curPos++;
    if (heavy[u] != -1) {
        decompose(heavy[u], h); // Continue heavy chain
    }
    for (int v : adj[u]) {
        if (v != parent[u] && v != heavy[u]) {
            decompose(v, v); // New chain
        }
    }
}

// Segment tree
void build(int idx, int l, int r) {
    if (l == r) {
        segtree[idx] = arr[l];
        return;
    }
    int mid = (l + r) / 2;
    build(idx * 2, l, mid);
    build(idx * 2 + 1, mid + 1, r);
    segtree[idx] = segtree[idx * 2] + segtree[idx * 2 + 1];
}

int query(int idx, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return 0;
    if (ql <= l && r <= qr) return segtree[idx];
    int mid = (l + r) / 2;
    return query(idx * 2, l, mid, ql, qr) +
           query(idx * 2 + 1, mid + 1, r, ql, qr);
}

void update(int idx, int l, int r, int ul, int ur, int val) {
    if (ul > r || ur < l) return;
    if (l == r) {
        segtree[idx] += val; // or = val for assignment
        return;
    }
    int mid = (l + r) / 2;
    update(idx * 2, l, mid, ul, ur, val);
    update(idx * 2 + 1, mid + 1, r, ul, ur, val);
    segtree[idx] = segtree[idx * 2] + segtree[idx * 2 + 1];
}

};

int main() {
    int n = 8;
    vector<vector<int>> adj(n);
    adj[0] = {1, 2, 3};
    adj[1] = {0, 4, 5};
    adj[2] = {0};
    adj[3] = {0};
    adj[4] = {1};
    adj[5] = {1, 6, 7};
    adj[6] = {5};

```

```

adj[7] = {5};

HLD hld(n, adj);
// Set initial values
for (int i = 0; i < n; i++) hld.setValue(i, i + 1);
hld.buildSegmentTree();

cout << "Sum 4->7: " << hld.pathQuery(3, 6) << "\n"; // nodes 3, 6: 4+7=11
hld.pathUpdate(3, 6, 10); // add 10 to all nodes on path 3->6
cout << "After update: " << hld.pathQuery(3, 6) << "\n"; // +10 per node

return 0;
}

```

Python Implementation

```

class HLD:
    def __init__(self, n, adj):
        self.n = n
        self.adj = adj
        self.parent = [-1] * n
        self.depth = [0] * n
        self.heavy = [-1] * n
        self.head = [0] * n
        self.pos = [0] * n
        self.sz = [0] * n
        self.arr = [0] * n
        self.curPos = 0

        self.dfs(0, -1)
        self.decompose(0, 0)

        self.segtree = [0] * (4 * n)

    def dfs(self, u, p):
        self.parent[u] = p
        self.sz[u] = 1
        max_sz = 0
        for v in self.adj[u]:
            if v == p:
                continue
            self.depth[v] = self.depth[u] + 1
            sub = self.dfs(v, u)
            self.sz[u] += sub
            if sub > max_sz:
                max_sz = sub
                self.heavy[u] = v
        return self.sz[u]

    def decompose(self, u, h):
        self.head[u] = h

```

```

self.pos[u] = self.curPos
self.curPos += 1
if self.heavy[u] != -1:
    self.decompose(self.heavy[u], h)
for v in self.adj[u]:
    if v != self.parent[u] and v != self.heavy[u]:
        self.decompose(v, v)

def set_value(self, node, val):
    self.arr[self.pos[node]] = val

def build(self):
    self._build(1, 0, self.n - 1)

def _build(self, idx, l, r):
    if l == r:
        self.segtree[idx] = self.arr[l]
        return
    mid = (l + r) // 2
    self._build(idx * 2, l, mid)
    self._build(idx * 2 + 1, mid + 1, r)
    self.segtree[idx] = self.segtree[idx * 2] + self.segtree[idx * 2 + 1]

def path_query(self, u, v):
    res = 0
    while self.head[u] != self.head[v]:
        if self.depth[self.head[u]] < self.depth[self.head[v]]:
            u, v = v, u
        res += self._query(1, 0, self.n - 1, self.pos[self.head[u]],
self.pos[u])
        u = self.parent[self.head[u]]
    if self.depth[u] > self.depth[v]:
        u, v = v, u
    res += self._query(1, 0, self.n - 1, self.pos[u], self.pos[v])
    return res

def _query(self, idx, l, r, ql, qr):
    if ql > r or qr < l:
        return 0
    if ql <= l and r <= qr:
        return self.segtree[idx]
    mid = (l + r) // 2
    return self._query(idx * 2, l, mid, ql, qr) + self._query(idx * 2 + 1, mid
+ 1, r, ql, qr)

n = 8
adj = [[] for _ in range(n)]
adj[0] = [1, 2, 3]
adj[1] = [0, 4, 5]
adj[2] = [0]
adj[3] = [0]
adj[4] = [1]
adj[5] = [1, 6, 7]
adj[6] = [5]

```

```
adj[7] = [5]

hld = HLD(n, adj)
for i in range(n):
    hld.set_value(i, i + 1)
hld.build()
print(hld.path_query(3, 6)) # 4 + 7 = 11
```

23.9 Complexity Analysis

Aspect	Complexity
Preprocessing	$O(n)$
Per Path Query	$O(\log^2 n)$ — $O(\log n)$ chains $\times$ $O(\log n)$ segment tree
Per Path Update	$O(\log^2 n)$
Space	$O(n)$

24. Centroid Decomposition

24.1 Overview

Centroid Decomposition recursively divides the tree by removing its **centroid** (a node whose removal splits the tree into subtrees each of size $\leq n/2$). This creates a **centroid tree** of height $O(\log n)$, enabling efficient path queries.

24.2 Intuition

Find a node such that all resulting components after removing it are at most half the original size. This node is the centroid. Recursively decompose each component. The resulting hierarchy is a tree of depth $O(\log n)$.

Analogy: Splitting a country into states. Find the most "central" city (centroid). Split into regions, find centroid of each region, and so on. The hierarchy has $O(\log n)$ levels.

24.3 When to Use It

- **Count paths** with certain properties (sum, length, value constraints)
- **Distance queries** in tree (closest, farthest, count with distance $\leq k$)
- **Tree DP** with path constraints that are hard to do in original tree
- Problems where $O(n \log n)$ per query is acceptable

24.4 When Not to Use It

- Tree is small — brute force $O(n^2)$ may be simpler
- Queries are simple (subtree sum, LCA) — simpler methods exist
- Need heavy path updates — HLD is better

24.5 Core Concepts

Concept	Definition
Centroid	Node whose removal gives components each $\leq n/2$
Centroid Tree	Tree formed by centroid decomposition (height $O(\log n)$)
Level	Depth in centroid tree

Key property: Every path in original tree between two nodes goes through the LCA of those nodes in the centroid tree.

24.6 Step-by-Step Algorithm

1. Find centroid of current component:
 - a. Compute subtree sizes via DFS
 - b. Find node where $\max_component_size \leq total_size / 2$
2. Make centroid the root of this level
3. Remove centroid (mark as processed)
4. Recursively decompose each neighbor's component
5. Connect centroid to centroids of subcomponents

24.7 Dry Run

Tree:



Total size = 5. Centroids: 1 or 2 (each max component = $3 \leq 2.5$? No! Let's compute properly.)

Compute sizes from any root:

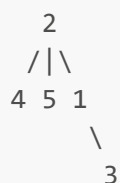
- If root = 1: sizes: 4(2), 3(3), 1(4), 1(5), 5(1) Max component after removing 1 = $\max(3, 3) = 3 \leq 2.5$? No.
- If root = 2: sizes from 2: 1(left=4), 1(right=5), 2(upper=1+3), 5(2) Removing 2: components sizes = 1, 1, 2 (3 if counting upper) $\rightarrow \max = 2 \leq 2.5$? Yes!

Centroid = 2.

Remove 2:

- Component 1: {4}, centroid = 4
- Component 2: {5}, centroid = 5
- Component 3: {1, 3}, centroid = 1 (size 2, max component = 1)

Centroid tree:



24.8 C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

class CentroidDecomp {
    int n;
    vector<vector<int>>> adj;
    vector<bool> removed;
    vector<int> sz;
    vector<int> parent; // parent in centroid tree

public:
    CentroidDecomp(int n, vector<vector<int>>>& adj)
        : n(n), adj(adj) {
        removed.assign(n, false);
        sz.assign(n, 0);
        parent.assign(n, -1);
        build(0, -1);
    }

    void build(int u, int p) {
        findSizes(u, -1);
        int c = findCentroid(u, -1, sz[u]);

        removed[c] = true;
        parent[c] = p;

        for (int v : adj[c]) {
            if (!removed[v]) {
                build(v, c);
            }
        }
    }

    vector<int> getParent() { return parent; }

private:
    void findSizes(int u, int p) {
        sz[u] = 1;
        for (int v : adj[u]) {
            if (v == p || removed[v]) continue;
            findSizes(v, u);
            sz[u] += sz[v];
        }
    }
};

```

```

    }
}

int findCentroid(int u, int p, int total) {
    for (int v : adj[u]) {
        if (v == p || removed[v]) continue;
        if (sz[v] > total / 2) {
            return findCentroid(v, u, total);
        }
    }
    return u;
}

};

// Example: Count paths with sum = target (uses centroid decomposition)
class PathCounter {
    int n;
    vector<vector<pair<int, int>>> adj; // (neighbor, weight)
    vector<bool> removed;
    vector<int> sz;
    long long ans = 0;
    int target;

public:
    PathCounter(int n, vector<vector<pair<int, int>>>& adj, int target)
        : n(n), adj(adj), target(target) {
        removed.assign(n, false);
        sz.assign(n, 0);
        solve(0);
    }

    long long getAns() { return ans; }

private:
    void solve(int u) {
        findSizes(u, -1);
        int c = findCentroid(u, -1, sz[u]);
        removed[c] = true;

        // Count paths that pass through centroid
        countPathsThrough(c);

        // Recurse on subcomponents
        for (auto [v, w] : adj[c]) {
            if (!removed[v]) solve(v);
        }
    }

    void findSizes(int u, int p) {
        sz[u] = 1;
        for (auto [v, w] : adj[u]) {
            if (v == p || removed[v]) continue;
            findSizes(v, u);
            sz[u] += sz[v];
        }
    }
};

```

```

    }
}

int findCentroid(int u, int p, int total) {
    for (auto [v, w] : adj[u]) {
        if (v == p || removed[v]) continue;
        if (sz[v] > total / 2) {
            return findCentroid(v, u, total);
        }
    }
    return u;
}

void countPathsThrough(int c) {
    unordered_map<int, int> distCount;
    distCount[0] = 1; // empty path from centroid itself

    for (auto [v, w] : adj[c]) {
        if (removed[v]) continue;

        // Collect distances from centroid in this subtree
        vector<pair<int, int>> dists; // (distance, node)
        getDistances(v, c, w, dists);

        // Count pairs where sum = target
        for (auto [d, node] : dists) {
            if (distCount.count(target - d)) {
                ans += distCount[target - d];
            }
        }

        // Add this subtree's distances to the global count
        for (auto [d, node] : dists) {
            distCount[d]++;
        }
    }
}

void getDistances(int u, int p, int dist, vector<pair<int, int>>& dists) {
    dists.push_back({dist, u});
    for (auto [v, w] : adj[u]) {
        if (v == p || removed[v]) continue;
        getDistances(v, u, dist + w, dists);
    }
}

};

int main() {
    int n = 5;
    vector<vector<pair<int, int>>> adj(n);
    adj[0] = {{1, 1}, {2, 2}};
    adj[1] = {{0, 1}, {3, 3}, {4, 4}};
    adj[2] = {{0, 2}};
    adj[3] = {{1, 3}};
}

```

```

adj[4] = {{1, 4}};

PathCounter pc(n, adj, 5);
cout << "Paths with sum 5: " << pc.getAns() << "\n";
return 0;
}

```

Python Implementation

```

class CentroidDecomp:
    def __init__(self, n, adj):
        self.n = n
        self.adj = adj
        self.removed = [False] * n
        self.sz = [0] * n
        self.parent = [-1] * n
        self._build(0, -1)

    def _build(self, u, p):
        self._find_sizes(u, -1)
        c = self._find_centroid(u, -1, self.sz[u])

        self.removed[c] = True
        self.parent[c] = p

        for v in self.adj[c]:
            if not self.removed[v]:
                self._build(v, c)

    def _find_sizes(self, u, p):
        self.sz[u] = 1
        for v in self.adj[u]:
            if v == p or self.removed[v]:
                continue
            self._find_sizes(v, u)
            self.sz[u] += self.sz[v]

    def _find_centroid(self, u, p, total):
        for v in self.adj[u]:
            if v == p or self.removed[v]:
                continue
            if self.sz[v] > total // 2:
                return self._find_centroid(v, u, total)
        return u

# Path counting example
class PathCounter:
    def __init__(self, n, adj, target):
        self.n = n
        self.adj = adj
        self.target = target

```

```

self.removed = [False] * n
self.sz = [0] * n
self.ans = 0
self._solve(0)

def _solve(self, u):
    self._find_sizes(u, -1)
    c = self._find_centroid(u, -1, self.sz[u])
    self.removed[c] = True

    self._count_paths_through(c)

    for v, w in self.adj[c]:
        if not self.removed[v]:
            self._solve(v)

def _find_sizes(self, u, p):
    self.sz[u] = 1
    for v, w in self.adj[u]:
        if v == p or self.removed[v]:
            continue
        self._find_sizes(v, u)
        self.sz[u] += self.sz[v]

def _find_centroid(self, u, p, total):
    for v, w in self.adj[u]:
        if v == p or self.removed[v]:
            continue
        if self.sz[v] > total // 2:
            return self._find_centroid(v, u, total)
    return u

def _count_paths_through(self, c):
    from collections import defaultdict
    dist_count = defaultdict(int)
    dist_count[0] = 1

    for v, w in self.adj[c]:
        if self.removed[v]:
            continue
        dists = []
        self._get_distances(v, c, w, dists)

        for d, node in dists:
            if self.target - d in dist_count:
                self.ans += dist_count[self.target - d]

        for d, node in dists:
            dist_count[d] += 1

def _get_distances(self, u, p, dist, dists):
    dists.append((dist, u))
    for v, w in self.adj[u]:
        if v == p or self.removed[v]:

```

```

        continue
    self._get_distances(v, u, dist + w, dists)

n = 5
adj = [[] for _ in range(n)]
adj[0] = [(1, 1), (2, 2)]
adj[1] = [(0, 1), (3, 3), (4, 4)]
adj[2] = [(0, 2)]
adj[3] = [(1, 3)]
adj[4] = [(1, 4)]

pc = PathCounter(n, adj, 5)
print(f"Paths with sum 5: {pc.getAns()}")

```

24.9 Complexity Analysis

Aspect	Complexity
Finding centroid	$O(n)$ per level
Total decomposition	$O(n \log n)$
Path counting per centroid	$O(n \log n)$ (with map)
Space	$O(n)$

24.10 Common Patterns

- **Count paths with sum/weight constraint:** Use centroid decomposition, process paths through centroid, combine from different subtrees using map
- **Closest pair / farthest pair:** Can be extended to distance-based queries
- **Tree DP with constraint:** Decompose, process centroid, recurse

24.11 Common Mistakes

- Forgetting to mark nodes as **removed** — leads to infinite recursion
- Computing sizes incorrectly inside removed subtrees
- Not handling the centroid itself when counting paths
- $O(n^2)$ if map operations are not careful

25. Euler Tour + Segment Tree

25.1 Overview

Euler Tour (also called **Flattening**) converts a tree into a **linear array** using DFS order. Combined with a **Segment Tree**, this enables **subtree queries and updates** in $O(\log n)$ time.

25.2 Intuition

When you perform a DFS on a tree, each node gets an **entry time** (tin) and an **exit time** (tout). All nodes in a subtree have their tin/tout within a contiguous range. This maps each subtree to a contiguous segment in an array.

Analogy: Taking a family photograph in birth order. Each person and all their descendants appear in a contiguous block of the photo.

25.3 When to Use It

- **Subtree sum/min/max queries**
- **Subtree updates** (add value to all nodes in subtree)
- **Path-to-root queries** (combine with Fenwick tree for point updates, path queries)
- **Tree flattening** + segment tree for range operations
- **LCA queries** (using RMQ on Euler tour array)

25.4 When Not to Use It

- Need **path queries** between arbitrary nodes — use HLD
- Tree is tiny — brute force is simpler
- No subtree operations — simple DFS is enough

25.5 Core Concepts

Concept	Explanation
tin[node]	Entry time during DFS (order of first visit)
tout[node]	Exit time during DFS (order after finishing subtree)
Euler array	Linear array where subtree of node u = range $[\text{tin}[u], \text{tout}[u]]$
Flat index	Position in the linear array = $\text{tin}[\text{node}]$

Property: Node v is in the subtree of u iff $\text{tin}[u] \leq \text{tin}[v] \leq \text{tout}[u]$.

25.6 Step-by-Step Algorithm

Preprocessing:

1. Run DFS from root, assign $\text{tin}[u]$ and $\text{tout}[u]$
2. Build array arr where $\text{arr}[\text{tin}[u]] = \text{value}[u]$
3. Build segment tree / Fenwick tree on arr

Subtree Query (sum of subtree of u):

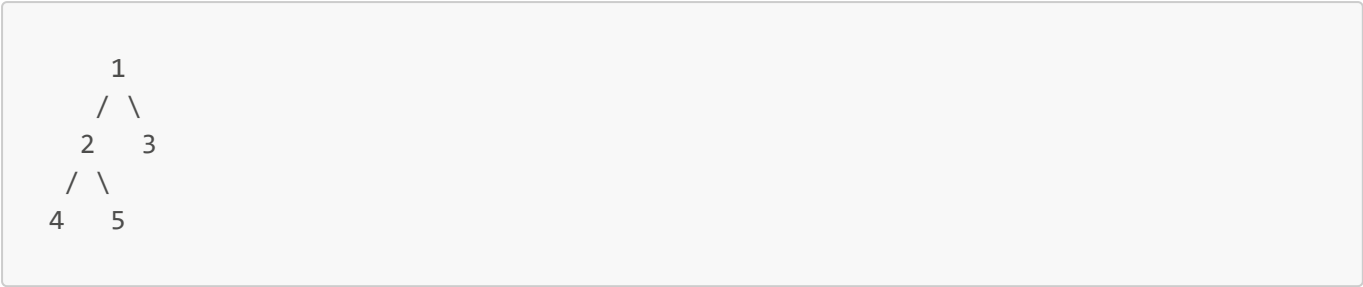
1. Query segment tree on range $[\text{tin}[u], \text{tout}[u]]$

Subtree Update (add val to all nodes in subtree of u):

1. Update segment tree on range $[\text{tin}[u], \text{tout}[u]]$ with $+\text{val}$

25.7 Dry Run

Tree:



DFS order (preorder): 1, 2, 4, 5, 3

Node	tin	tout
1	0	4
2	1	3
3	4	4
4	2	2
5	3	3

Subtree ranges:

- Subtree of 1: [0, 4] — entire array
- Subtree of 2: [1, 3] — nodes 2, 4, 5
- Subtree of 4: [2, 2] — just node 4

Array (values): [v1, v2, v4, v5, v3] at positions 0, 1, 2, 3, 4

Sum of subtree 2 = sum(arr[1..3]) = v2 + v4 + v5

25.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class EulerTourSegTree {
    int n, timer;
    vector<vector<int>> adj;
    vector<int> tin, tout;
    vector<long long> segtree;
    vector<long long> arr; // values at flat positions

public:
    EulerTourSegTree(int n, vector<vector<int>>& adj, vector<int>& values)
        : n(n), adj(adj) {
        timer = 0;
        tin.assign(n, 0);
        tout.assign(n, 0);
        arr.assign(n, 0);
    }
};
```

```

    dfs(0, -1, values);

    segtree.assign(4 * n, 0);
    build(1, 0, n - 1);
}

void dfs(int u, int p, vector<int>& values) {
    tin[u] = timer;
    arr[timer] = values[u];
    timer++;

    for (int v : adj[u]) {
        if (v != p) {
            dfs(v, u, values);
        }
    }

    tout[u] = timer - 1;
}

// Subtree sum query
long long querySubtree(int u) {
    return query(1, 0, n - 1, tin[u], tout[u]);
}

// Subtree update (add val)
void updateSubtree(int u, long long val) {
    update(1, 0, n - 1, tin[u], tout[u], val);
}

// Point update (set value at node u)
void updatePoint(int u, long long val) {
    update(1, 0, n - 1, tin[u], tin[u], val); // For assignment, rebuild
    // For range add, use lazy propagation
}

private:
void build(int idx, int l, int r) {
    if (l == r) {
        segtree[idx] = arr[l];
        return;
    }
    int mid = (l + r) / 2;
    build(idx * 2, l, mid);
    build(idx * 2 + 1, mid + 1, r);
    segtree[idx] = segtree[idx * 2] + segtree[idx * 2 + 1];
}

long long query(int idx, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return 0;
    if (ql <= l && r <= qr) return segtree[idx];
    int mid = (l + r) / 2;
    return query(idx * 2, l, mid, ql, qr) +

```

```

        query(idx * 2 + 1, mid + 1, r, ql, qr);
    }

    void update(int idx, int l, int r, int ul, int ur, long long val) {
        if (ul > r || ur < l) return;
        if (l == r) {
            segtree[idx] = val;
            return;
        }
        int mid = (l + r) / 2;
        update(idx * 2, l, mid, ul, ur, val);
        update(idx * 2 + 1, mid + 1, r, ul, ur, val);
        segtree[idx] = segtree[idx * 2] + segtree[idx * 2 + 1];
    }
};

// LCA using Euler tour + RMQ (Segment tree for min depth)
class LCA_Euler {
    int n;
    vector<vector<int>>> adj;
    vector<int> euler, depth, first, segtree;

public:
    LCA_Euler(int n, vector<vector<int>>>& adj) : n(n), adj(adj) {
        first.assign(n, -1);
        depth.assign(n, 0);
        dfs(0, -1, 0);

        segtree.assign(4 * euler.size(), 0);
        build(1, 0, euler.size() - 1);
    }

    void dfs(int u, int p, int d) {
        first[u] = euler.size();
        euler.push_back(u);
        depth[u] = d;

        for (int v : adj[u]) {
            if (v != p) {
                dfs(v, u, d + 1);
                euler.push_back(u);
            }
        }
    }

    int lca(int u, int v) {
        int l = min(first[u], first[v]);
        int r = max(first[u], first[v]);
        return euler[query(1, 0, euler.size() - 1, l, r)];
    }

private:
    void build(int idx, int l, int r) {
        if (l == r) {

```

```

        segtree[idx] = 1;
        return;
    }
    int mid = (l + r) / 2;
    build(idx * 2, l, mid);
    build(idx * 2 + 1, mid + 1, r);

    int left = segtree[idx * 2];
    int right = segtree[idx * 2 + 1];
    segtree[idx] = (depth[euler[left]] < depth[euler[right]]) ? left : right;
}

int query(int idx, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return -1;
    if (ql <= l && r <= qr) return segtree[idx];
    int mid = (l + r) / 2;
    int left = query(idx * 2, l, mid, ql, qr);
    int right = query(idx * 2 + 1, mid + 1, r, ql, qr);
    if (left == -1) return right;
    if (right == -1) return left;
    return (depth[euler[left]] < depth[euler[right]]) ? left : right;
}

};

int main() {
    int n = 5;
    vector<vector<int>> adj(n);
    adj[0] = {1, 2};
    adj[1] = {0, 3, 4};
    adj[2] = {0};
    adj[3] = {1};
    adj[4] = {1};

    vector<int> values = {10, 20, 30, 40, 50};

    EulerTourSegTree et(n, adj, values);
    cout << "Sum of subtree 1: " << et.querySubtree(1) << "\n"; // 20+40+50 = 110

    LCA_Euler lca(n, adj);
    cout << "LCA(3,4): " << lca.lca(3, 4) << "\n"; // 1

    return 0;
}

```

Python Implementation

```

class EulerTourSegTree:
    def __init__(self, n, adj, values):
        self.n = n
        self.adj = adj
        self.timer = 0

```

```

self.tin = [0] * n
self.tout = [0] * n
self.arr = [0] * n

self._dfs(0, -1, values)
self.segtree = [0] * (4 * n)
self._build(1, 0, n - 1)

def _dfs(self, u, p, values):
    self.tin[u] = self.timer
    self.arr[self.timer] = values[u]
    self.timer += 1
    for v in self.adj[u]:
        if v != p:
            self._dfs(v, u, values)
    self.tout[u] = self.timer - 1

def query_subtree(self, u):
    return self._query(1, 0, self.n - 1, self.tin[u], self.tout[u])

def update_subtree(self, u, val):
    self._update(1, 0, self.n - 1, self.tin[u], self.tout[u], val)

def _build(self, idx, l, r):
    if l == r:
        self.segtree[idx] = self.arr[l]
        return
    mid = (l + r) // 2
    self._build(idx * 2, l, mid)
    self._build(idx * 2 + 1, mid + 1, r)
    self.segtree[idx] = self.segtree[idx * 2] + self.segtree[idx * 2 + 1]

def _query(self, idx, l, r, ql, qr):
    if ql > r or qr < l:
        return 0
    if ql <= l and r <= qr:
        return self.segtree[idx]
    mid = (l + r) // 2
    return self._query(idx * 2, l, mid, ql, qr) + self._query(idx * 2 + 1, mid
+ 1, r, ql, qr)

def _update(self, idx, l, r, ul, ur, val):
    if ul > r or ur < l:
        return
    if l == r:
        self.segtree[idx] = val
        return
    mid = (l + r) // 2
    self._update(idx * 2, l, mid, ul, ur, val)
    self._update(idx * 2 + 1, mid + 1, r, ul, ur, val)
    self.segtree[idx] = self.segtree[idx * 2] + self.segtree[idx * 2 + 1]

n = 5
adj = [[] for _ in range(n)]

```

```

adj[0] = [1, 2]
adj[1] = [0, 3, 4]
adj[2] = [0]
adj[3] = [1]
adj[4] = [1]
values = [10, 20, 30, 40, 50]
et = EulerTourSegTree(n, adj, values)
print(et.query_subtree(1)) # 110

```

25.9 Complexity Analysis

Aspect	Complexity
Preprocessing (DFS)	$O(n)$
Subtree query	$O(\log n)$
Subtree update	$O(\log n)$ with lazy propagation
Point update	$O(\log n)$
Space	$O(n)$

25.10 Common Patterns

- **Subtree queries:** Use Euler tour flattening + segment tree
- **LCA using RMQ:** Euler tour + segment tree for min depth
- **Path-to-root queries:** Use Fenwick tree (add at node, query prefix)
- **Subtree with lazy propagation:** Add to subtree, query subtree

25.11 Edge Cases

- Single node: $tin = tout = 0$
- Path queries between arbitrary nodes: not supported directly (use HLD)
- Large depth: recursion limit may need to be increased

26. Link-Cut Tree

26.1 Overview

Link-Cut Tree (LCT) is a dynamic tree data structure that supports **link, cut, path queries, and path updates** in $O(\log n)$ amortized time. It uses **splay trees** as the underlying balance mechanism and represents the tree as a collection of **preferred paths**.

26.2 Intuition

LCT maintains a forest of trees that can change dynamically. It decomposes each tree into **preferred paths** (similar to HLD but dynamic) and represents each path as a splay tree keyed by depth. The **expose** operation makes a root-to-node path a single preferred path, which can then be queried/updated.

Analogy: A road network with temporary express lanes. When you need to travel from point A to point B, you temporarily create a highway (expose) that connects them, do your business, and the highway dissolves back.

26.3 When to Use It

- **Dynamic tree** with edge insertions (link) and deletions (cut)
- **Path queries** on dynamic trees (min, max, sum between two nodes)
- **Path updates** on dynamic trees
- **Root changes** (evert/makeRoot)
- **Dynamic connectivity** (forest with link/cut)

26.4 When Not to Use It

- Tree is **static** — use HLD or Euler tour (simpler, faster constants)
- Only need **subtree** queries — use Euler tour + segment tree
- Small constraint ($n < 1000$) — brute force is simpler
- No link/cut operations — simpler data structures exist
- Implementation complexity is a concern — LCT is the most complex tree DS

26.5 Core Concepts

Concept	Explanation
Splay tree	Self-adjusting BST used as the building block
Preferred path	A path in the original tree represented as one splay tree
Expose	Make the path from root to node a single preferred path (splay tree)
MakeRoot	Make a node the root of its tree (evert)
Link	Connect two trees by adding an edge
Cut	Disconnect an edge, splitting the tree
Virtual tree	The collection of all splay trees (one per preferred path)
Preferred child	Child that is on the same preferred path as parent (last accessed)

26.6 Step-by-Step Algorithm

Key Operations:

Splay(x): Rotate x to the root of its splay tree
 - Standard splay tree rotations (zig, zig-zig, zig-zag)

Access(x): Make root-to-x path preferred (expose)

1. Last = NULL
2. While x != NULL:
 - a. Splay(x)
 - b. x->right = last (change preferred child)
 - c. Update(x)

```
d. Last = x, x = path-parent(x)
```

```
MakeRoot(x):
```

1. Access(x)
2. Splay(x)
3. Reverse the subtree (flip to change depth order)

```
FindRoot(x):
```

1. Access(x)
2. Splay(x)
3. Go leftmost (smallest depth)

```
Link(x, y): Connect two trees
```

1. MakeRoot(x)
2. Access(y), Splay(y)
3. If FindRoot(x) != FindRoot(y): set x->parent = y

```
Cut(x, y): Disconnect edge
```

1. MakeRoot(x)
2. Access(y), Splay(y)
3. If y->left == x and x->right == NULL: y->left = NULL, x->parent = NULL

```
PathQuery(x, y):
```

1. MakeRoot(x)
2. Access(y)
3. Splay(y) — now y's subtree in splay tree has info for path x→y

26.7 Dry Run (simple example)

Initial tree:

```

  1
 / \
2   3
```

Link(4, 2):

1. MakeRoot(4) — 4 is now root of its own tree
2. Access(2) — expose path root→2 in tree
3. Link 4 as child of 2

Result:

```

  1
 / \
2   3
|
4
```

Cut(2, 1):

1. MakeRoot(2) — 2 becomes root
2. Access(1), Splay(1) — expose path
3. Check: 1->left == 2? In the splay tree representation, after MakeRoot(2), Access(1)...
4. Remove connection

Result:

```

Tree 1:      Tree 2:
  1           2
 / \        / \
3  (2      4 (1... wait this isn't right)
    removed)

```

26.8 C++ Implementation (Simplified LCT)

```

#include <bits/stdc++.h>
using namespace std;

struct Node {
    int val, sum, lazy;
    bool rev;
    Node *ch[2], *par;

    Node(int v = 0) : val(v), sum(v), lazy(0), rev(false) {
        ch[0] = ch[1] = par = NULL;
    }
};

class LinkCutTree {
public:
    bool isRoot(Node* x) {
        return !x->par || (x->par->ch[0] != x && x->par->ch[1] != x);
    }

    void push(Node* x) {
        if (x->rev) {
            swap(x->ch[0], x->ch[1]);
            if (x->ch[0]) x->ch[0]->rev ^= true;
            if (x->ch[1]) x->ch[1]->rev ^= true;
            x->rev = false;
        }
        if (x->lazy) {
            if (x->ch[0]) { x->ch[0]->val += x->lazy; x->ch[0]->sum += x->lazy; x->ch[0]->lazy += x->lazy; }
            if (x->ch[1]) { x->ch[1]->val += x->lazy; x->ch[1]->sum += x->lazy; x->ch[1]->lazy += x->lazy; }
            x->lazy = 0;
        }
    }
};

```

```
}

void update(Node* x) {
    x->sum = x->val;
    if (x->ch[0]) x->sum += x->ch[0]->sum;
    if (x->ch[1]) x->sum += x->ch[1]->sum;
}

void rotate(Node* x) {
    Node* p = x->par;
    Node* g = p->par;
    int dir = (p->ch[1] == x);

    if (!isRoot(p)) g->ch[g->ch[1] == p] = x;
    x->par = g;

    p->ch[dir] = x->ch[dir ^ 1];
    if (x->ch[dir ^ 1]) x->ch[dir ^ 1]->par = p;

    x->ch[dir ^ 1] = p;
    p->par = x;

    update(p);
    update(x);
}

void splay(Node* x) {
    vector<Node*> st;
    Node* y = x;
    st.push_back(y);
    while (!isRoot(y)) { y = y->par; st.push_back(y); }
    while (!st.empty()) { push(st.back()); st.pop_back(); }

    while (!isRoot(x)) {
        Node* p = x->par;
        Node* g = p->par;
        if (!isRoot(p)) {
            if ((g->ch[0] == p) == (p->ch[0] == x))
                rotate(p);
            else
                rotate(x);
        }
        rotate(x);
    }
}

void access(Node* x) {
    Node* last = NULL;
    for (Node* y = x; y; y = y->par) {
        splay(y);
        y->ch[1] = last;
        update(y);
        last = y;
    }
}
```

```

        splay(x);
    }

    void makeRoot(Node* x) {
        access(x);
        x->rev ^= true;
    }

    Node* findRoot(Node* x) {
        access(x);
        while (x->ch[0]) { push(x); x = x->ch[0]; }
        splay(x);
        return x;
    }

    void link(Node* x, Node* y) {
        makeRoot(x);
        if (findRoot(y) != x) x->par = y;
    }

    void cut(Node* x, Node* y) {
        makeRoot(x);
        access(y);
        if (y->ch[0] == x && !x->ch[1]) {
            y->ch[0] = NULL;
            x->par = NULL;
            update(y);
        }
    }

    int pathQuery(Node* x, Node* y) {
        makeRoot(x);
        access(y);
        return y->sum;
    }

    void pathUpdate(Node* x, Node* y, int val) {
        makeRoot(x);
        access(y);
        y->val += val;
        y->sum += val;
        y->lazy += val;
    }
};

int main() {
    LinkCutTree lct;

    Node* n1 = new Node(10);
    Node* n2 = new Node(20);
    Node* n3 = new Node(30);
    Node* n4 = new Node(40);

    // Build tree: 1-2-3, then link 4 to 2

```

```

lct.link(n1, n2);
lct.link(n2, n3);
lct.link(n4, n2);

cout << "Sum 1->3: " << lct.pathQuery(n1, n3) << "\n"; // 10+20+30 = 60

lct.pathUpdate(n1, n4, 5);
cout << "After update, sum 1->4: " << lct.pathQuery(n1, n4) << "\n";

// Cut edge between 2 and 3
lct.cut(n2, n3);
cout << "After cut, root of 3: " << lct.findRoot(n3)->val << "\n";

return 0;
}

```

Python Implementation

```

class Node:
    def __init__(self, val=0):
        self.val = val
        self.sum = val
        self.lazy = 0
        self.rev = False
        self.ch = [None, None]
        self.par = None

class LinkCutTree:
    def is_root(self, x):
        return not x.par or (x.par.ch[0] != x and x.par.ch[1] != x)

    def push(self, x):
        if x.rev:
            x.ch[0], x.ch[1] = x.ch[1], x.ch[0]
            if x.ch[0]: x.ch[0].rev ^= True
            if x.ch[1]: x.ch[1].rev ^= True
            x.rev = False
        if x.lazy:
            for c in [x.ch[0], x.ch[1]]:
                if c:
                    c.val += x.lazy
                    c.sum += x.lazy
                    c.lazy += x.lazy
            x.lazy = 0

    def update(self, x):
        x.sum = x.val
        if x.ch[0]: x.sum += x.ch[0].sum
        if x.ch[1]: x.sum += x.ch[1].sum

    def rotate(self, x):

```

```

    p = x.par
    g = p.par
    d = 1 if p.ch[1] == x else 0

    if not self.is_root(p):
        g.ch[1 if g.ch[1] == p else 0] = x
    x.par = g

    p.ch[d] = x.ch[d ^ 1]
    if x.ch[d ^ 1]: x.ch[d ^ 1].par = p

    x.ch[d ^ 1] = p
    p.par = x

    self.update(p)
    self.update(x)

def splay(self, x):
    st = []
    y = x
    st.append(y)
    while not self.is_root(y):
        y = y.par
        st.append(y)
    while st:
        self.push(st.pop())

    while not self.is_root(x):
        p = x.par
        g = p.par
        if not self.is_root(p):
            if (g.ch[0] == p) == (p.ch[0] == x):
                self.rotate(p)
            else:
                self.rotate(x)
        self.rotate(x)

def access(self, x):
    last = None
    y = x
    while y:
        self.splay(y)
        y.ch[1] = last
        self.update(y)
        last = y
        y = y.par
    self.splay(x)

def make_root(self, x):
    self.access(x)
    x.rev ^= True

def find_root(self, x):
    self.access(x)

```

```

        while x.ch[0]:
            self.push(x)
            x = x.ch[0]
        self.splay(x)
        return x

    def link(self, x, y):
        self.make_root(x)
        if self.find_root(y) != x:
            x.par = y

    def cut(self, x, y):
        self.make_root(x)
        self.access(y)
        if y.ch[0] == x and not x.ch[1]:
            y.ch[0] = None
            x.par = None
            self.update(y)

    def path_query(self, x, y):
        self.make_root(x)
        self.access(y)
        return y.sum

    def path_update(self, x, y, val):
        self.make_root(x)
        self.access(y)
        y.val += val
        y.sum += val
        y.lazy += val

# Example
lct = LinkCutTree()
n1 = Node(10)
n2 = Node(20)
n3 = Node(30)
n4 = Node(40)

lct.link(n1, n2)
lct.link(n2, n3)
lct.link(n4, n2)

print(lct.path_query(n1, n3)) # 60
lct.path_update(n1, n4, 5)
print(lct.path_query(n1, n4)) # 10+25+40=75
lct.cut(n2, n3)
print(lct.find_root(n3).val) # 30 (its own root now)

```

26.9 Complexity Analysis

Operation	Complexity
Access (Expose)	$O(\log n)$ amortized
MakeRoot	$O(\log n)$ amortized
FindRoot	$O(\log n)$ amortized
Link	$O(\log n)$ amortized
Cut	$O(\log n)$ amortized
Path Query	$O(\log n)$ amortized
Path Update	$O(\log n)$ amortized
Space	$O(n)$

26.10 When to Use

- Dynamic tree with link/cut operations
- Need to query/update path between any two nodes in a changing tree
- Competitive programming problem specifies "link" and "cut" operations

26.11 Common Mistakes

- Not calling `push()` before accessing child pointers
- Not calling `update()` after changing children
- Confusing `splay()` with `access()` — access includes splay at the end
- Forgetting `makeRoot` before `link` or path operations
- Incorrect `isRoot()` check (must check both children)

26.12 Edge Cases

- Linking nodes already connected (check with `findRoot` first)
- Cutting non-existent edge
- Single node tree
- Forest with multiple disconnected trees

Quick Reference: Related Data Structures

DS	Static/Dynamic	Subtree Ops	Path Ops	Link/Cut	Complexity
Euler Tour + SegTree	Static	✓	✗ (only root→node)	✗	$O(\log n)$
HLD + SegTree	Static	✓	✓	✗	$O(\log^2 n)$
Centroid Decomp	Static	✗	✓ (counting)	✗	$O(\log n)$ (queries)
Binary Lifting	Static	✗	✗ (LCA only)	✗	$O(\log n)$ / query

DS	Static/Dynamic	Subtree Ops	Path Ops	Link/Cut	Complexity
Link-Cut Tree	Dynamic	✗	✓	✓	$O(\log n)$ amortized

20. Final Cheat Sheet

Binary Tree

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

Traversals

Type	Order	Use Case
Preorder	Root → Left → Right	Serialization, copying
Inorder	Left → Root → Right	Sorted order (BST)
Postorder	Left → Right → Root	Deletion, bottom-up DP
Level Order	BFS	Shortest path, level-based

Key Formulas

Property	Formula
Height	$1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$
Diameter	$\max(\text{diameter}, \text{leftH} + \text{rightH})$ at each node
Balanced	`
LCA	If both sides return non-null → root; else return non-null side
BST Property	$\text{min} < \text{root->val} < \text{max}$ for range-based validation

Complexities

Algorithm	Time	Space
Traversal (recursive)	$O(n)$	$O(h)$
Traversal (iterative)	$O(n)$	$O(h)$
Morris Traversal	$O(n)$	$O(1)$

Algorithm	Time	Space
BST Search/Insert/Delete	$O(\log n)$ avg, $O(n)$ worst	$O(h)$
Binary Lifting (preprocess)	$O(n \log n)$	$O(n \log n)$
Binary Lifting (query)	$O(\log n)$	-
HLD (query)	$O(\log^2 n)$	$O(n)$
Centroid Decomposition	$O(n \log n)$	$O(n)$
Euler Tour + SegTree	$O(\log n)$ per query	$O(n)$
Link-Cut Tree	$O(\log n)$ amortized	$O(n)$

Common Traps

- ❌ BST validation: checking only immediate children (must check ancestors via range)
- ❌ Diameter always passes through root (it might not — check all nodes)
- ❌ Balanced tree only checks root (must check all nodes)
- ❌ Recursion overflow for deep trees (use iterative or Morris)
- ❌ Forgetting backtracking in root-to-leaf paths (pop after recursion)
- ❌ Not handling NULL/empty tree in base cases
- ❌ Wrong successor during BST deletion (minimum of right subtree)

Must-Know for Interviews

1. **Traversals**: Recursive + iterative (stack/queue) + Morris (bonus)
2. **LCA**: Simple recursive $O(n)$ solution
3. **Diameter + Balanced**: Combine with height calculation
4. **BST validation**: Range propagation (min/max parameters) or inorder check
5. **Serialize/Deserialize**: Preorder with NULL markers
6. **Construct tree**: Preorder + inorder → rebuild recursively
7. **Level order**: BFS with queue, level separation via size tracking
8. **Path sum**: Decrement target, check at leaf
9. **Root-to-leaf paths**: Backtracking with DFS
10. **Vertical/Boundary traversal**: Map-based column grouping + separate boundary collection

Happy coding! 🌳